



L'Institut Supérieur d'Informatique, de Modélisation et de leurs
Applications (ISIMA)
Clermont Auvergne INP

Rapport du Projet de Fin d'Année

Filière : Optimisation et Intelligence Artificielle (F4)

Application de l'Apprentissage par Renforcement Profond au Problème du Voyageur de Commerce

L2GLS — Learning to Guide Local Search

Réalisé par :

Yassir MASFOUR

Encadré par :

Pr. Sylvie NORRE

Pr. Nathalie GRANGEON

Année Universitaire 2025–2026

Résumé

Ce rapport présente une étude sur l'application de l'apprentissage par renforcement profond à la résolution du problème du voyageur de commerce (TSP). L'approche proposée, L2GLS (*Learning to Guide Local Search*), consiste à entraîner un agent d'apprentissage par renforcement pour piloter une procédure de recherche locale guidée (GLS). L'agent décide, à chaque itération, de l'opérateur de recherche locale à appliquer et du nombre de pénalisations GLS à effectuer. Nous reproduisons la méthode originale fondée sur l'algorithme REINFORCE, puis proposons deux contributions : l'utilisation d'un Deep Q-Network (DQN) et d'un algorithme Proximal Policy Optimization (PPO) comme alternatives. Un espace d'actions hiérarchique à deux niveaux est introduit pour le PPO, décomposant la décision en choix du nombre de pénalités puis choix de l'opérateur.

Abstract

This report investigates the application of deep reinforcement learning to the Travelling Salesman Problem (TSP). The proposed approach, L2GLS (Learning to Guide Local Search), trains a reinforcement learning agent to control a Guided Local Search (GLS) procedure. At each iteration, the agent selects which local search operator to apply and how many GLS penalty updates to perform. We reproduce the original REINFORCE method, then propose two contributions : a Deep Q-Network (DQN) and a Proximal Policy Optimization (PPO) algorithm with a hierarchical two-level action space.

Table des matières

Résumé	2
Abstract	3
Introduction générale	7
1 Le Problème du Voyageur de Commerce	9
1.1 Notations et conventions	9
1.2 Formulation du problème	10
1.3 Représentation dans le plan euclidien	11
1.4 État de l’art	11
2 Recherche Locale et Guided Local Search	14
2.1 Principes de la recherche locale	14
2.2 Opérateur 2-Opt	17
2.3 Opérateur Relocate	18
2.4 Opérateur Swap	18
2.5 Opérateur 3-Opt	19
2.6 Opérateur Three-Permutation	19
2.7 Limites de la recherche locale	20
2.8 Guided Local Search (GLS)	20
2.8.1 Origines et idée fondatrice	20
2.8.2 Features et fonction de coût augmentée	21
2.8.3 Fonction d’utilité et mécanisme de pénalisation	21
2.8.4 Algorithme GLS	22
2.8.5 Fast Local Search (FLS)	23
2.8.6 GLS appliquée au TSP	23

2.8.7	Le degré de liberté restant	24
2.8.8	Axes de recherche : pénalités continues	24
3	Apprentissage par Renforcement profond	25
3.1	Fondements du deep learning	25
3.1.1	Neurone artificiel	25
3.1.2	Fonctions d'activation	26
3.1.3	Réseau de neurones multicouche	26
3.1.4	Mécanisme d'attention et Transformer	27
3.2	Apprentissage par Renforcement : fondements	28
3.2.1	Processus de décision de Markov	28
3.2.2	REINFORCE	30
3.2.3	Deep Q-Network (DQN)	31
3.2.4	Proximal Policy Optimization (PPO)	33
3.2.5	Synthèse comparative	35
4	L2GLS — Formalisation et Contributions	36
4.1	Architecture générale	36
4.2	Formulation MDP pour le TSP	37
4.2.1	Espace d'états $\mathcal{S}$	37
4.2.2	Espace d'actions $\mathcal{A}$	37
4.2.3	Fonction de transition T	38
4.2.4	Signal de récompense R	38
4.3	L2GLS avec REINFORCE (méthode originale)	39
4.3.1	Architecture du réseau	39
4.3.2	Procédure d'entraînement	40
4.3.3	Algorithme complet L2GLS (inférence)	40
4.4	Contribution 1 : L2GLS avec DQN	40
4.5	Contribution 2 : L2GLS avec PPO et espace hiérarchique	41
4.5.1	Justification du PPO pour L2GLS	41
4.5.2	Architecture du réseau	41
4.5.3	Entraînement par curriculum	41
4.5.4	Hyperparamètres	42
4.6	Implémentation logicielle	42

5	Protocole expérimental et résultats	44
5.1	Configuration matérielle et logicielle	44
5.2	Configuration d'entraînement	44
5.3	Instances de test	46
5.4	Métriques d'évaluation	46
5.5	Méthodes comparées	47
5.6	Résultats sur instances aléatoires	47
5.7	Résultats sur instances TSPLIB et comparaison avec Concorde	49
5.7.1	Comparaison directe avec Concorde	51
5.7.2	Comparaison avec L2GLS original et L2I	51
5.8	Analyse de la sous-performance du PPO	52
5.9	Distribution des opérateurs	53
5.10	Synthèse	54
	Conclusion et perspectives	55
	Fiche de suivi des compétences	57
	Annexe : Diagramme de Gantt	61

Introduction générale

Imaginons un livreur qui doit desservir un ensemble de villes en parcourant la distance la plus courte possible, puis revenir à son point de départ. Ce problème, d'apparence simple, est en réalité l'un des problèmes d'optimisation combinatoire les plus difficiles que l'on connaisse : c'est le problème du voyageur de commerce, ou TSP (*Travelling Salesman Problem*). Sa difficulté tient au fait que le nombre de tournées possibles croît de façon factorielle avec le nombre de villes : pour 20 villes, il existe déjà plus de 10^{17} tournées distinctes. Il est donc impossible d'énumérer toutes les solutions pour en trouver la meilleure.

Face à cette difficulté, deux familles d'approches se sont développées. D'une part, les méthodes exactes garantissent de trouver la meilleure solution, mais leur temps de calcul croît exponentiellement. D'autre part, les métaheuristiques (recherche locale, recuit simulé, algorithmes génétiques) produisent de bonnes solutions en un temps raisonnable, sans garantie d'optimalité. Un problème commun à toutes les méthodes de recherche locale est la convergence vers un *minimum local* : un point où aucune petite modification ne permet d'améliorer la solution, bien que celle-ci puisse être très loin de l'optimum global.

La *Guided Local Search* (GLS), introduite par Voudouris et Tsang [2], apporte une solution élégante : elle modifie dynamiquement la fonction de coût par un mécanisme de pénalités, guidant la recherche hors des minima locaux. Cependant, la GLS laisse un degré de liberté important : à chaque étape, quel opérateur de recherche locale faut-il appliquer, et combien de pénalisations faut-il effectuer ? Ce choix, traditionnellement laissé à l'expertise humaine, constitue le point de départ de notre étude. La question centrale est : *peut-on automatiser ce choix ?*

C'est l'idée de la méthode L2GLS (*Learning to Guide Local Search*), proposée par Sultana et al. [1] : entraîner un agent d'apprentissage par renforcement à piloter la procédure GLS. L'article original utilise l'algorithme REINFORCE [4]. Notre travail se structure autour de trois axes : la reproduction de la méthode originale, puis deux contributions consistant à remplacer REINFORCE par un Deep Q-Network (DQN) et par un algorithme Proximal Policy Optimization (PPO) [5] avec un espace d'actions hiérarchique à deux niveaux.

Ce rapport est organisé comme suit. Le chapitre 1 pose le cadre : nous y définissons

le TSP, les notations et les méthodes de résolution existantes. Le chapitre 2 détaille les opérateurs de recherche locale et la GLS. Le chapitre 3 expose les fondements de l'apprentissage par renforcement. Le chapitre 4 formalise L2GLS et nos contributions. Le chapitre 5 présente les expérimentations.

Chapitre 1

Le Problème du Voyageur de Commerce

Ce chapitre pose les bases de tout le rapport. Nous commençons par établir les notations mathématiques utilisées dans l'ensemble du document, puis nous formulons rigoureusement le problème du voyageur de commerce, et enfin nous présentons un panorama des méthodes de résolution existantes. Ce panorama permet de comprendre pourquoi les métaheuristiques, et en particulier la combinaison de la GLS avec l'apprentissage par renforcement, constituent une voie de recherche pertinente.

1.1 Notations et conventions

Afin de garantir la clarté des développements qui suivent, nous fixons les notations utilisées dans tout le rapport.

On note $\mathbb{N}$ l'ensemble des entiers naturels (avec $0 \in \mathbb{N}$), $\mathbb{Z}$ l'ensemble des entiers relatifs, $\mathbb{R}$ l'ensemble des nombres réels, et $\mathbb{R}^+$ l'ensemble des réels positifs ou nuls ($\mathbb{R}^+ = \{x \in \mathbb{R} \mid x \geq 0\}$). Pour tout entier $n \geq 1$, on note $\mathbb{R}^n$ l'espace vectoriel des vecteurs à n composantes réelles, et $\mathbb{R}^{n \times m}$ l'espace des matrices à n lignes et m colonnes à coefficients réels.

Les scalaires sont notés par des lettres minuscules ou des lettres grecques (n, λ, α). Les vecteurs sont notés par des lettres minuscules en gras : $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{R}^n$. Les matrices sont notées par des lettres majuscules : pour toute matrice A , l'élément en position (i, j) est noté a_{ij} . Les ensembles sont notés par des lettres majuscules calligraphiques : $\mathcal{S}, \mathcal{A}, \mathcal{F}$.

Définition 1.1 (Permutation). Pour tout entier $n \geq 1$, on note $\mathcal{S}_n$ l'ensemble de toutes les permutations de $\{1, 2, \dots, n\}$. Une permutation $\pi \in \mathcal{S}_n$ est une bijection de $\{1, \dots, n\}$ dans lui-même : elle associe à chaque entier i un entier $\pi(i)$, de sorte que chaque valeur apparaît exactement une fois. Le nombre de permutations distinctes est $|\mathcal{S}_n| = n!$.

Cette notion de permutation est centrale dans notre travail : comme nous allons le voir,

une tournée du voyageur de commerce est entièrement déterminée par une permutation des villes.

1.2 Formulation du problème

Le TSP demande de trouver, parmi toutes les tournées possibles, celle de coût minimal. Pour formaliser cette idée, nous avons besoin de la notion de graphe.

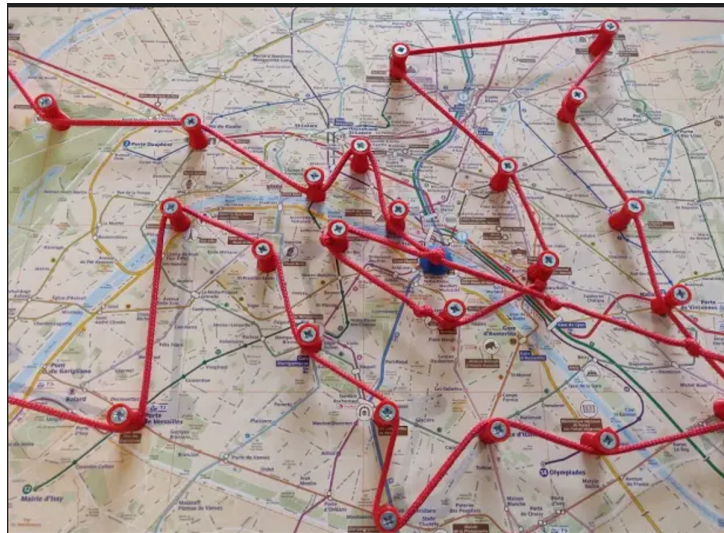


FIGURE 1.1 – Exemple d’une tournée du voyageur de commerce sur 10 villes.

Définition 1.2 (Graphe complet). Un graphe $G = (V, E)$ est constitué d’un ensemble de sommets V et d’un ensemble d’arêtes $E \subseteq \{\{v_i, v_j\} \mid v_i, v_j \in V, v_i \neq v_j\}$. Le graphe est dit *complet* lorsque chaque paire de sommets distincts est reliée par une arête : $|E| = n(n-1)/2$ où $n = |V|$.

Dans le TSP, les sommets représentent les villes et les arêtes les chemins entre villes. Le graphe est complet car on suppose que le voyageur peut se rendre directement de n’importe quelle ville à n’importe quelle autre. Chaque arête $\{v_i, v_j\}$ est associée à un coût $c(v_i, v_j) \in \mathbb{R}^+$. Nous considérons le TSP *symétrique* : $c(v_i, v_j) = c(v_j, v_i)$ pour toutes les paires de villes.

Définition 1.3 (Tournée — cycle hamiltonien). Une tournée est un parcours qui visite chaque ville exactement une fois et revient à la ville de départ. En théorie des graphes, on parle de *cycle hamiltonien*. Une tournée est entièrement déterminée par une permutation $\pi \in \mathcal{S}_n$: on visite les villes dans l’ordre $\pi(1), \pi(2), \dots, \pi(n)$, puis on revient à $\pi(1)$.

Nous pouvons maintenant formuler le TSP de manière rigoureuse.

Définition 1.4 (Problème du voyageur de commerce — TSP). Soit $G = (V, E)$ un graphe complet avec $|V| = n$ et $c : E \rightarrow \mathbb{R}^+$. Le TSP consiste à trouver une permutation $\pi^* \in \mathcal{S}_n$ telle que :

$$\pi^* = \arg \min_{\pi \in \mathcal{S}_n} \sum_{i=1}^n c(\pi(i), \pi(i \bmod n + 1)), \quad (1.1)$$

où l'opération $i \bmod n + 1$ assure que la dernière ville $\pi(n)$ est reliée à la première $\pi(1)$, fermant le cycle.

Le nombre de cycles hamiltoniens distincts est $(n - 1)!/2$. Le facteur $1/2$ vient de la symétrie : parcourir le cycle dans un sens ou dans l'autre donne le même coût. Cette croissance factorielle rend l'énumération exhaustive impraticable : pour $n = 20$, il y a environ 6×10^{16} cycles distincts.

Remarque 1.1. Le TSP est un problème *NP-difficile*. Cela signifie qu'il n'existe pas d'algorithme connu capable de résoudre toutes les instances en un temps polynomial en n .

1.3 Représentation dans le plan euclidien

Dans notre travail, chaque ville est un point du plan représenté par ses coordonnées $(x, y) \in \mathbb{R}^2$. On dispose d'un ensemble de N villes :

$$C = \{(x_\ell, y_\ell) \mid \ell = 1, \dots, N\} \subset \mathbb{R}^2. \quad (1.2)$$

Nous définissons la matrice de distances $D = [d_{ij}] \in \mathbb{R}^{N \times N}$, où d_{ij} est la distance entre les villes c_i et c_j . La définition de la distance est laissée ouverte pour la généralité ; Dans la majorité des cas, nous utilisons la distance euclidienne : $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

Une tournée est représentée par $\pi = (\pi_1, \pi_2, \dots, \pi_N)$, et son coût est :

$$g(\pi) = \sum_{i=1}^N d(\pi_i, \pi_{i+1}), \quad \text{avec } \pi_{N+1} = \pi_1. \quad (1.3)$$

Cette fonction g est la fonction objectif que nous cherchons à minimiser.

1.4 État de l'art

Les méthodes de résolution du TSP se classent en plusieurs catégories. Chaque catégorie résout certaines limitations de la précédente, et cette progression motive notre approche.

Méthodes exactes. Le solveur Concorde Concorde (Applegate et al., 2006) état de l’art pour les méthodes exactes, utilise des techniques de *branch-and-cut* et de programmation linéaire pour trouver la solution optimale d’instances allant jusqu’à plusieurs milliers de villes. Cependant, le temps de calcul reste exponentiel dans le pire cas. C’est cette limitation qui motive les approches approchées.

Heuristiques constructives. L’heuristique du plus proche voisin (*Nearest Neighbor*) construit une tournée en ajoutant à chaque étape la ville non visitée la plus proche. L’heuristique gloutonne (*Greedy*) ajoute les arêtes par ordre de coût croissant. Ces méthodes sont rapides mais fournissent surtout un point de départ pour les méthodes d’amélioration.

Métaheuristiques. La recherche locale itérée, le recuit simulé, la recherche tabou [10] et les algorithmes génétiques hybrides améliorent itérativement une solution. Parmi ces approches, la GLS [2] modifie la fonction objectif elle-même pour guider la recherche. Nous la détaillerons au chapitre 2.

Méthodes fondées sur l’apprentissage. Le modèle *Attention Model* (AM) de Kool et al. [7] utilise un mécanisme d’attention de type Transformer : le réseau génère la tournée ville par ville, en apprenant à pondérer l’importance de chaque ville non visitée en fonction du contexte courant. AM est entraîné par REINFORCE [4] avec une baseline *greedy rollout* et atteint des gaps de l’ordre de 2–3% sur TSP100. POMO (*Policy Optimization with Multiple Optima*) améliore cette approche en explorant simultanément plusieurs points de départ pour la construction de la tournée, réduisant le gap à environ 1% sur TSP100. Les *Pointer Networks* de Vinyals et al. introduisent le mécanisme de pointage qui permet au réseau de désigner directement un élément de l’entrée comme sortie, une idée fondatrice pour l’application des réseaux de neurones aux problèmes combinatoires. L’approche REINFORCE de Bello et al. [6] étend les Pointer Networks avec un entraînement par gradient de politique sans supervision. Cependant, ces méthodes purement neuronales peinent à rivaliser avec les métaheuristiques bien paramétrées sur les grandes instances ($N > 100$), car elles tentent de résoudre le TSP *directement* par un réseau de neurones, sans exploiter les algorithmes d’optimisation classiques qui ont fait leurs preuves depuis des décennies.

Vers la combinaison. C’est à l’intersection de l’optimisation classique et de l’apprentissage que se situe L2GLS [1]. L’approche L2I (*Learning to Improve*), proposée par Lu et al. [9], entraîne un agent RL à sélectionner itérativement parmi un ensemble d’opérateurs de perturbation et d’amélioration pour résoudre des problèmes de routage. L’agent L2I agit comme un *contrôleur* qui choisit, à chaque itération, quelle perturbation appliquer puis quelle procédure d’amélioration utiliser. Cette approche a montré des résultats

prometteurs, mais reste limitée par l'absence de mécanisme structuré d'échappement des minima locaux : l'agent perturbe la solution de manière ad hoc, sans modifier la fonction objectif. L2GLS [1] va plus loin en intégrant le mécanisme de pénalisation de la GLS comme composant structurel de l'environnement RL. L'agent ne se contente pas de choisir un opérateur : il décide également du niveau de diversification (nombre de pénalisations GLS), ce qui lui permet de contrôler explicitement le compromis entre intensification (améliorer la solution courante) et diversification (échapper aux minima locaux). Cette intégration GLS–RL constitue la différence fondamentale avec L2I et justifie le choix de L2GLS comme cadre de notre étude. Pour comprendre ce pilotage, il faut d'abord comprendre les outils à disposition : les opérateurs de recherche locale et le mécanisme GLS. C'est l'objet du chapitre suivant.

Chapitre 2

Recherche Locale et Guided Local Search

Le chapitre précédent a formulé le TSP et montré que les méthodes exactes sont impraticables pour les grandes instances. Ce chapitre présente les outils qui permettent d'améliorer progressivement une solution : les opérateurs de recherche locale [8]. Nous verrons d'abord le principe général, puis les quatre opérateurs spécifiques utilisés dans L2GLS, et enfin le mécanisme GLS qui permet de dépasser la limitation fondamentale de la recherche locale : la convergence vers un minimum local.

2.1 Principes de la recherche locale

L'intuition de la recherche locale est simple : partant d'une solution existante (par exemple, celle obtenue par l'heuristique du plus proche voisin), on explore les solutions “voisines” — c'est-à-dire obtenues par une petite modification de la solution courante — et on se déplace vers une meilleure solution voisine. On répète ce processus jusqu'à ce qu'aucun voisin ne soit meilleur que la solution courante.

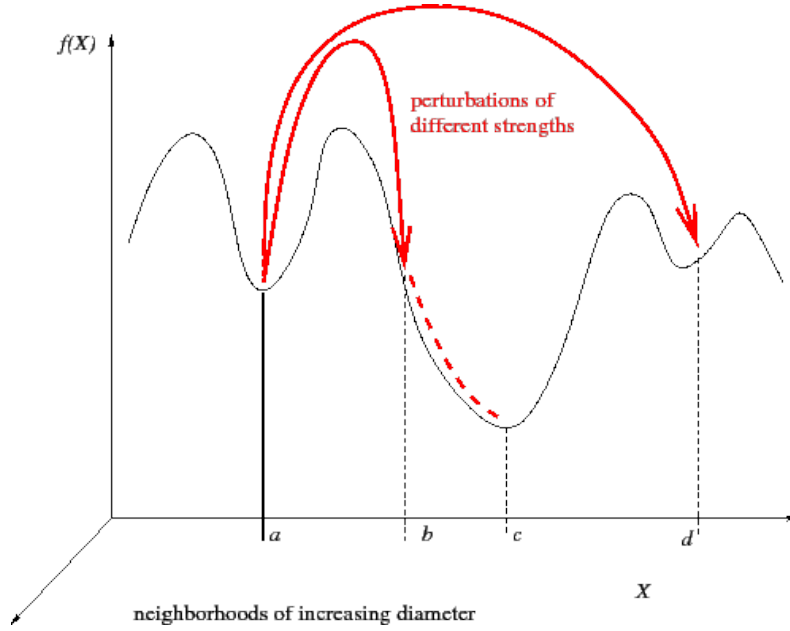


FIGURE 2.1 – Illustration du principe de la recherche locale itérée : exploration du voisinage et convergence vers un minimum local.

Pour formaliser cette idée, nous avons besoin de définir ce qu’est un “voisinage” [2] d’une solution.

Définition 2.1 (Voisinage). Soit $\mathcal{S}$ l’ensemble de toutes les solutions réalisables d’un problème d’optimisation (dans notre cas, l’ensemble de toutes les tournées du TSP), et soit $g : \mathcal{S} \rightarrow \mathbb{R}$ la fonction objectif à minimiser. Un *voisinage* est une application $\mathcal{N} : \mathcal{S} \rightarrow 2^{\mathcal{S}}$ qui associe à chaque solution s un sous-ensemble $\mathcal{N}(s) \subseteq \mathcal{S}$ de solutions atteignables par une transformation élémentaire, appelée *mouvement* (ou *move*).

La nature du voisinage $\mathcal{N}$ dépend du type de mouvement considéré. Pour le TSP, un mouvement peut consister à inverser un segment de la tournée (opérateur 2-opt), à déplacer une ville par l’opérateur relocate [8], ou à échanger deux villes par opérateur swap [8]. Chacun de ces opérateurs définit un voisinage différent, et donc une vision différente de ce que signifie “modifier légèrement” une solution. Nous les détaillerons dans les sections suivantes.

Définition 2.2 (Minimum local). Une solution $s \in \mathcal{S}$ est un *minimum local* de g par rapport au voisinage $\mathcal{N}$ si aucune solution voisine n’est strictement meilleure :

$$g(s) \leq g(y), \quad \forall y \in \mathcal{N}(s). \quad (2.1)$$

La recherche locale remplace itérativement la solution courante s par une solution $y \in \mathcal{N}(s)$ telle que $g(y) < g(s)$, jusqu’à atteindre un minimum local. Deux stratégies de sélection existent : la stratégie *best improvement* examine tout le voisinage et choisit

la meilleure solution ; la stratégie *first improvement* accepte le premier voisin améliorant trouvé.

Algorithm 1: Recherche locale générique

Entrée: Solution initiale $s_0 \in \mathcal{S}$, voisinage $\mathcal{N}$, fonction objectif g

Sortie: Minimum local s^*

```

1  $s \leftarrow s_0$ ;
2 repeat
3    $s_{\text{best}} \leftarrow \arg \min_{y \in \mathcal{N}(s)} g(y)$ ;
4   if  $g(s_{\text{best}}) < g(s)$  then  $s \leftarrow s_{\text{best}}$ ;
5 until  $g(s_{\text{best}}) \geq g(s)$ ;
6 return  $s$ 

```

Remarque 2.1 (Problème fondamental). Un minimum local n'est pas nécessairement un optimum global. La qualité du minimum local dépend du choix de l'opérateur (qui détermine le voisinage), de la solution initiale, et de l'ordre d'exploration. Ce problème est la motivation principale de la GLS, présentée en section 2.8.

Nous présentons maintenant les quatre opérateurs utilisés dans L2GLS. Pour chaque opérateur, nous donnons d'abord l'intuition du mouvement qu'il effectue sur la tournée, puis sa définition formelle, le calcul du gain associé (qui permet de déterminer si un mouvement est améliorant sans recalculer le coût total), et sa complexité. Notons que dans tous ces algorithmes, la matrice de distances D peut être remplacée par la matrice augmentée $D' = D + \lambda P$ [2] dans le cadre de la GLS (section 2.8.6), sans aucune modification structurelle des opérateurs.

2.2 Opérateur 2-Opt

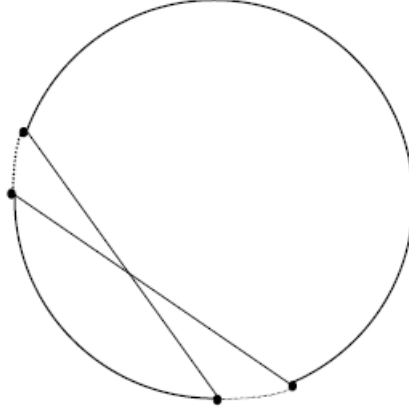


FIGURE 2.2 – Illustration du mouvement 2-opt : suppression de deux arêtes et reconnexion du segment inversé.

L'opérateur 2-Opt introduit par Croes (1958) est le mouvement de recherche locale le plus classique pour le TSP. L'intuition est la suivante : on considère deux arêtes non adjacentes de la tournée, on les supprime (ce qui “coupe” la tournée en deux segments), puis on reconnecte les segments en inversant l'un d'entre eux. Si la nouvelle tournée est plus courte que l'ancienne, le mouvement est accepté.

Définition 2.3 (Mouvement 2-opt). Soit $\pi = (\pi_1, \dots, \pi_n)$ une tournée et $1 \leq i < j \leq n$ deux indices. Le mouvement $2\text{opt}(\pi, i, j)$ supprime les arêtes (π_i, π_{i+1}) et (π_j, π_{j+1}) et produit la nouvelle tournée :

$$\pi' = (\pi_1, \dots, \pi_i, \pi_j, \pi_{j-1}, \dots, \pi_{i+1}, \pi_{j+1}, \dots, \pi_n), \quad (2.2)$$

c'est-à-dire que le segment $(\pi_{i+1}, \dots, \pi_j)$ est inversé.

Le gain associé mesure la différence de coût entre la nouvelle tournée et l'ancienne, *sans* recalculer le coût total (ce qui serait en $O(n)$ à chaque évaluation). En effet, seules les arêtes autour des points de coupure changent :

$$\Delta g_{2\text{-opt}}(i, j) = d(\pi_i, \pi_j) + d(\pi_{i+1}, \pi_{j+1}) - d(\pi_i, \pi_{i+1}) - d(\pi_j, \pi_{j+1}). \quad (2.3)$$

Un mouvement est améliorant si et seulement si $\Delta g_{2\text{-opt}}(i, j) < 0$. L'évaluation de tous les couples (i, j) possibles est en $O(n^2)$.

2.3 Opérateur Relocate

L'opérateur Relocate introduit par Or (1976) explore un type de voisinage complémentaire au 2-opt. Plutôt que d'inverser un segment, il retire une ville de sa position courante et la réinsère ailleurs dans la tournée. L'intuition est qu'une ville peut être mal placée : ses voisins actuels dans la tournée sont loin d'elle, mais elle serait plus "à sa place" entre deux autres villes proches.

Définition 2.4 (Mouvement Relocate). Soit $\pi = (\pi_1, \dots, \pi_n)$ une tournée et deux indices $i \neq j$. Le mouvement Relocate(π, i, j) retire la ville π_i et l'insère entre π_j et π_{j+1} :

$$\pi' = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_j, \pi_i, \pi_{j+1}, \dots, \pi_n). \quad (2.4)$$

Le gain est :

$$\Delta g_{\text{rel}}(i, j) = [d(\pi_{i-1}, \pi_{i+1}) + d(\pi_j, \pi_i) + d(\pi_i, \pi_{j+1})] - [d(\pi_{i-1}, \pi_i) + d(\pi_i, \pi_{i+1}) + d(\pi_j, \pi_{j+1})]. \quad (2.5)$$

La complexité d'une passe complète est en $O(n^2)$.

2.4 Opérateur Swap

L'opérateur Swap introduit par [8] échange les positions de deux villes dans la tournée. L'intuition est que deux villes peuvent être mutuellement mieux placées si elles échangent leurs positions respectives dans le parcours.

Définition 2.5 (Mouvement Swap). Pour deux indices $i < j$ avec $j > i + 1$ (cas non adjacent), le mouvement Swap(π, i, j) échange π_i et π_j :

$$\pi' = (\dots, \pi_{i-1}, \pi_j, \pi_{i+1}, \dots, \pi_{j-1}, \pi_i, \pi_{j+1}, \dots). \quad (2.6)$$

On note : $d_{\text{new}} = [d(\pi_{i-1}, \pi_j) + d(\pi_j, \pi_{i+1}) + d(\pi_{j-1}, \pi_i) + d(\pi_i, \pi_{j+1})]$

$d_{\text{old}} = [d(\pi_{i-1}, \pi_i) + d(\pi_i, \pi_{i+1}) + d(\pi_{j-1}, \pi_j) + d(\pi_j, \pi_{j+1})]$.

Le gain est :

$$\Delta g_{\text{swap}}(i, j) = d_{\text{new}} - d_{\text{old}} \quad (2.7)$$

La complexité d'une passe complète est en $O(n^2)$.

2.5 Opérateur 3-Opt

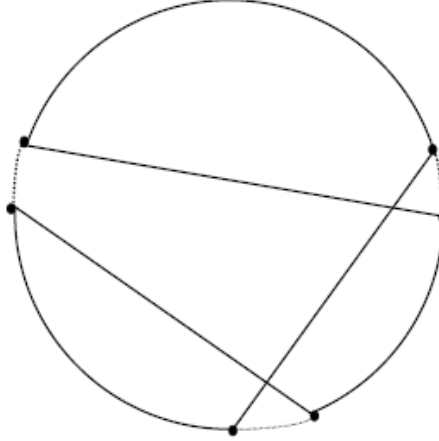


FIGURE 2.3 – Illustration du mouvement 3-opt : suppression de trois arêtes et reconnexion optimale des segments.

Le 3-opt introduit par Lin (1965) généralise le 2-opt en supprimant *trois* arêtes, produisant trois segments qui peuvent être reconnectés de différentes manières, y compris en inversant certains segments. L'intuition : le 3-opt explore un voisinage plus riche que le 2-opt (tout mouvement 2-opt est un cas particulier de 3-opt), au prix d'une complexité accrue.

Définition 2.6 (Mouvement 3-opt). Soient $1 \leq i < j < k \leq n$. Le mouvement $3opt(\pi, i, j, k)$ supprime les arêtes (π_i, π_{i+1}) , (π_j, π_{j+1}) et (π_k, π_{k+1}) , produisant trois segments $A = (\pi_1, \dots, \pi_i)$, $B = (\pi_{i+1}, \dots, \pi_j)$, $C = (\pi_{j+1}, \dots, \pi_k)$ et la queue $D = (\pi_{k+1}, \dots, \pi_n)$. Parmi les $2^3 - 1 = 7$ reconnections non triviales (chaque segment pouvant être inversé ou non), le mouvement retient celle de coût minimal.

Le gain dépend du type de reconnexion $r \in \{1, \dots, 7\}$:

$$\Delta g_{3-opt}(i, j, k, r) = \sum_{\text{nouvelles arêtes}} d - [d(\pi_i, \pi_{i+1}) + d(\pi_j, \pi_{j+1}) + d(\pi_k, \pi_{k+1})]. \quad (2.8)$$

Complexité : $O(n^3)$ par passe, ce qui est nettement plus coûteux que le 2-opt. Cependant, le 3-opt peut trouver des améliorations que le 2-opt ne peut pas détecter, car il explore un voisinage strictement plus grand.

2.6 Opérateur Three-Permutation

L'opérateur Three-Permutation introduit par [1], spécifique au cadre L2GLS, offre un compromis entre diversité des mouvements et coût de calcul. L'intuition est la suivante :

on sélectionne trois villes aléatoires dans la tournée, on considère les $3! = 6$ façons de réarranger ces trois villes dans leurs positions respectives, et on retient la permutation de coût minimal. Ce mouvement peut modifier jusqu’à six arêtes simultanément, ce qui lui confère plus de diversité que le swap (qui modifie au plus quatre arêtes), tout en restant beaucoup moins coûteux que le 3-opt complet (qui est en $O(n^3)$).

Définition 2.7 (Mouvement Three-Permutation). Soient trois indices distincts i_1, i_2, i_3 tirés uniformément dans $\{1, \dots, n\}$. Le mouvement $3\text{-Perm}(\pi, i_1, i_2, i_3)$ retient la permutation $\sigma^* \in \mathcal{S}_3$ des trois villes $\{\pi_{i_1}, \pi_{i_2}, \pi_{i_3}\}$ qui minimise le coût total :

$$\sigma^* = \arg \min_{\sigma \in \mathcal{S}_3} g(\pi \text{ avec } (\pi_{i_1}, \pi_{i_2}, \pi_{i_3}) \leftarrow \sigma(\pi_{i_1}, \pi_{i_2}, \pi_{i_3})). \quad (2.9)$$

2.7 Limites de la recherche locale

Chacun des quatre opérateurs précédents, utilisé seul, converge inévitablement vers un minimum local (définition 2.2). La qualité de ce minimum dépend du choix de l’opérateur, de l’ordre d’exploration du voisinage, et de la solution initiale. Or, un minimum local peut être très éloigné de l’optimum global : la recherche locale est “piégée” dans un creux de la fonction objectif g , sans moyen d’en sortir.

Ce constat pose deux questions. D’abord, comment échapper aux minima locaux ? La GLS, présentée dans la section suivante, répond à cette question en modifiant la fonction objectif. Ensuite, quel opérateur utiliser à chaque étape ? Différents opérateurs explorent différentes parties du voisinage, et leur efficacité varie selon les instances et l’état de la recherche. C’est cette seconde question qui sera résolue par l’apprentissage par renforcement au chapitre 4.

2.8 Guided Local Search (GLS)

2.8.1 Origines et idée fondatrice

La GLS, introduite par Voudouris et Tsang [2], généralise le mécanisme de pénalisation de l’architecture GENET [3], originalement conçue pour la résolution de problèmes de satisfaction de contraintes (CSP). Le terme “réseau de neurones” dans ce contexte ne désigne pas un réseau profond au sens du deep learning, mais une architecture dont la structure encode les contraintes du problème et dont la règle de mise à jour des poids permet d’échapper aux minima locaux. L’idée fondatrice est élégante : plutôt que de modifier le processus de recherche (comme le recuit simulé qui accepte des détériorations aléatoires), ou de mémoriser les solutions visitées comme la recherche tabou, la GLS modifie la *fonction objectif elle-même*. En ajoutant des pénalités à certaines composantes de la

solution, on modifie le paysage d’optimisation de telle sorte que le minimum local courant cesse d’en être un, forçant la recherche locale à explorer de nouvelles régions.

Pour mettre en œuvre cette idée, nous avons besoin de deux concepts : les *features* (quelles composantes de la solution pénaliser) et la fonction d’*utilité* (comment choisir quelle feature pénaliser).

2.8.2 Features et fonction de coût augmentée

Les features sont des propriétés de la solution que l’on peut pénaliser. Le choix des features est spécifique au problème : pour le TSP, les features sont les arêtes présentes dans la tournée (nous le formaliserons en section 2.8.6).

Définition 2.8 (Feature et fonction indicatrice). Soit $\mathcal{F} = \{f_1, \dots, f_M\}$ un ensemble de M features. Pour chaque feature f_i , on définit :

1. une *fonction indicatrice* $I_i : \mathcal{S} \rightarrow \{0, 1\} : I_i(s) = 1$ si f_i est présente dans la solution s , 0 sinon ;
2. un *coût* $c_i \in \mathbb{R}^+$: l’impact de la feature f_i sur la qualité de la solution ;
3. une *pénalité* $p_i \in \mathbb{N}$, initialisée à 0 et incrémentée au cours de l’algorithme.

Les pénalités permettent de modifier la fonction objectif. Plus une feature a été pénalisée, plus son coût “apparent” augmente, ce qui pousse la recherche locale à éviter les solutions qui la contiennent.

Définition 2.9 (Fonction de coût augmentée). Étant donné un paramètre $\lambda > 0$ contrôlant l’intensité de la pénalisation, la fonction de coût augmentée est :

$$h(s) = g(s) + \lambda \cdot \sum_{i=1}^M p_i \cdot I_i(s). \quad (2.10)$$

Le point clé est le suivant : la recherche locale minimise h (et non g), mais l’évaluation de la qualité d’une solution se fait toujours par g [2]. Ainsi, la GLS guide la recherche en modifiant le paysage d’optimisation via h , tout en évaluant la qualité réelle des solutions via g . Le paramètre λ contrôle l’agressivité de la pénalisation : une valeur élevée permet d’échapper rapidement aux minima locaux, mais peut conduire à des oscillations ; une valeur faible favorise une exploration plus prudente.

2.8.3 Fonction d’utilité et mécanisme de pénalisation

Il reste à déterminer *quelle* feature pénaliser à chaque itération. L’intuition est la suivante : on souhaite pénaliser les features qui sont à la fois coûteuses (elles contribuent

beaucoup au coût de la solution) et peu pénalisées (elles n'ont pas encore été suffisamment explorées). La fonction d'utilité formalise cette intuition.

Définition 2.10 (Fonction d'utilité). Pour une solution s et une feature f_i , la fonction d'utilité est :

$$\text{utility}(s, f_i) = I_i(s) \cdot \frac{c_i}{1 + p_i}. \quad (2.11)$$

L'utilité d'une feature dépend de trois facteurs : sa présence dans la solution courante ($I_i(s)$, car il est inutile de pénaliser une feature absente), son coût (c_i , car les features coûteuses sont prioritaires), et l'inverse de sa pénalité accumulée ($1/(1+p_i)$, car les features déjà souvent pénalisées sont moins prioritaires). À chaque itération, la feature d'utilité maximale voit sa pénalité incrémentée :

$$i^* = \arg \max_{i \in \{1, \dots, M\}} \text{utility}(s, f_i), \quad p_{i^*} \leftarrow p_{i^*} + 1. \quad (2.12)$$

Ce mécanisme garantit que les features coûteuses et fréquentes sont davantage pénalisées, ce qui modifie la surface du paysage via h et force la recherche locale à explorer de nouvelles régions. Le minimum local de g n'étant généralement pas un minimum local de h [2], la recherche peut se poursuivre.

2.8.4 Algorithme GLS

Algorithm 2: Guided Local Search (GLS)

Entrée: Instance $(\mathcal{S}, g, \mathcal{F})$, paramètre λ , critère d'arrêt

Sortie: Meilleure solution s^* par rapport à g

```

1 Générer une solution initiale  $s_0 \in \mathcal{S}$ ;
2  $p_i \leftarrow 0$  pour tout  $i \in \{1, \dots, M\}$ ;
3  $s \leftarrow \text{LocalSearch}(s_0, h)$ ;
4  $s^* \leftarrow s$ ;
5 while critère d'arrêt non satisfait do
6    $i^* \leftarrow \arg \max_i \text{utility}(s, f_i)$  ; // Feature d'utilité max.
7    $p_{i^*} \leftarrow p_{i^*} + 1$  ; // Pénaliser
8   Mettre à jour  $h$  selon l'éq. (2.10);
9    $s \leftarrow \text{LocalSearch}(s, h)$  ; // Nouveau min. local de  $h$ 
10  if  $g(s) < g(s^*)$  then  $s^* \leftarrow s$ ;
11 end
12 return  $s^*$ 

```

2.8.5 Fast Local Search (FLS)

La FLS (*Fast Local Search*) [2] est une technique d'accélération complémentaire. L'idée est de décomposer le voisinage $\mathcal{N}$ en m sous-voisinages $\mathcal{N}_1, \dots, \mathcal{N}_m$, chacun associé à un bit d'activation $b_k \in \{0, 1\}$.

Définition 2.11 (Voisinage actif). Un sous-voisinage $\mathcal{N}_k$ est actif si $b_k = 1$. Le voisinage actif est :

$$\mathcal{N}^{\text{act}}(s) = \bigcup_{\{k|b_k=1\}} \mathcal{N}_k(s). \quad (2.13)$$

Lorsqu'un sous-voisinage est exploré sans trouver de mouvement améliorant, il est désactivé ($b_k \leftarrow 0$). Lorsqu'un mouvement améliorant est trouvé, les sous-voisinages affectés sont réactivés. La combinaison GLS + FLS (GFLS) permet d'éviter le recalcul complet du voisinage après chaque pénalisation, accélérant considérablement la recherche sur les grandes instances.

2.8.6 GLS appliquée au TSP

Pour le TSP, l'application de la GLS se spécialise naturellement : les features sont les arêtes de la tournée.

Définition 2.12 (Features TSP). Pour une tournée π , la feature associée à l'arête (c_i, c_j) a pour fonction indicatrice :

$$I_{e_{ij}}(\pi) = \begin{cases} 1 & \text{si l'arête } (c_i, c_j) \text{ est présente dans } \pi, \\ 0 & \text{sinon.} \end{cases} \quad (2.14)$$

Le coût de la feature est la distance de l'arête : $c_{ij} = d_{ij}$.

Chaque arête (c_i, c_j) est associée à une pénalité p_{ij} . La matrice de pénalités symétrique $P = [p_{ij}]$ est initialisée à zéro. La fonction de coût augmentée devient :

$$h(\pi) = g(\pi) + \lambda \cdot \sum_{i < j} I_{e_{ij}}(\pi) \cdot p_{ij}. \quad (2.15)$$

En pratique, on définit la matrice de distances augmentée $D' = D + \lambda \cdot P$. La recherche locale utilise D' au lieu de D pour évaluer les mouvements : aucun des algorithmes (2-opt, relocate, swap, three-permutation) présentés précédemment n'a besoin d'être modifié structurellement. Il suffit de remplacer d_{ij} par $d'_{ij} = d_{ij} + \lambda \cdot p_{ij}$.

La fonction d'utilité pour le TSP est :

$$U(\pi, e_{ij}) = I_{e_{ij}}(\pi) \cdot \frac{d_{ij}}{1 + p_{ij}}. \quad (2.16)$$

Le paramètre λ est défini par $\lambda = \alpha \cdot g(\pi_{nn}^*)/N$ [gls*original*], $og(\pi_{nn}^*)$ est le coût de la solution du plus proche voisin, N le nombre de villes, et α un coefficient (fixé à 0,3 dans L2GLS [1]).

2.8.7 Le degré de liberté restant

Observons que la GLS ne prescrit pas le choix de l’opérateur de recherche locale. L’algorithme 2 appelle `LocalSearch( $s, h$ )` sans préciser *quel* opérateur utiliser : 2-opt ? relocate ? swap ? three-permutation ? De plus, le nombre de pénalisations GLS par itération n’est pas non plus fixé : faut-il pénaliser une, deux, ou trois features avant de relancer la recherche locale ?

Ce double degré de liberté — choix de l’opérateur et nombre de pénalisations — est précisément ce que l’agent d’apprentissage par renforcement va apprendre à contrôler. Pour comprendre comment, nous devons d’abord présenter les fondements de l’apprentissage par renforcement. C’est l’objet du chapitre suivant.

2.8.8 Axes de recherche : pénalités continues

Dans la GLS standard, les pénalités p_i sont des entiers incrémentés par pas de 1 (éq. (2.12)). Un axe de recherche potentiel consisterait à utiliser des pénalités continues $p_i \in \mathbb{R}^+$, mises à jour par un incrément proportionnel à l’utilité : $p_{i*} \leftarrow p_{i*} + \eta \cdot \text{utility}(s, f_{i*})$, où $\eta > 0$ est un pas d’apprentissage. Cette approche permettrait un contrôle plus fin de la pénalisation et pourrait être couplée avec un apprentissage automatique du pas η par l’agent RL. Nous n’explorons pas cette direction dans ce travail, mais elle constitue une perspective intéressante.

Une autre piste de recherche prometteuse consiste à traiter la fonction h comme une fonction continue, en l’intégrant directement comme terme de perte au sein du réseau de neurones, à l’image des réseaux de type PINN (Physics-Informed Neural Networks). Cette approche soulève néanmoins la question de la discrétisation : le passage entre l’espace continu, dans lequel h est définie, et l’espace discret, sur lequel opère le réseau, introduit inévitablement des erreurs d’approximation. Ce compromis constitue en lui-même un sujet d’étude à part entière, que nous ne traitons pas dans ce travail.

Chapitre 3

Apprentissage par Renforcement profond

Le chapitre précédent a mis en évidence un double degré de liberté dans la GLS : le choix de l'opérateur de recherche locale et le nombre de pénalisations. Ce chapitre présente le cadre formel qui permet d'automatiser ce choix : l'apprentissage par renforcement profond (DRL, pour *Deep Reinforcement Learning*). Le DRL est par définition la combinaison de l'apprentissage par renforcement et l'apprentissage profond. Nous commençons par les fondements du deep learning car les trois algorithmes RL utilisent des réseaux de neurones profonds pour approximer la politique ou la fonction de valeur. Nous présentons ensuite le formalisme de l'apprentissage par renforcement (MDP), puis les trois algorithmes : REINFORCE, DQN, PPO. Pour chaque algorithme, nous expliquons pourquoi il est pertinent pour notre problème.

3.1 Fondements du deep learning

Un réseau de neurones est un modèle paramétrique qui apprend à approximer une fonction inconnue à partir de données. Nous en avons besoin car l'espace d'états de notre problème est immense ($s \in \mathbb{R}^{N \times (9+2H)}$) : il est impossible de stocker ou d'énumérer toutes les valeurs possibles. Le réseau de neurones fournit une approximation compacte et généralisable.

3.1.1 Neurone artificiel

Définition 3.1 (Neurone artificiel). Un neurone artificiel est une fonction $f : \mathbb{R}^d \rightarrow \mathbb{R}$ définie par :

$$f(\mathbf{x}) = \sigma \left(\sum_{j=1}^d w_j x_j + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (3.1)$$

où $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$ est l'entrée, $\mathbf{w} = (w_1, \dots, w_d) \in \mathbb{R}^d$ les *poids*, $b \in \mathbb{R}$ le *biais*, et $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ la *fonction d'activation* (non linéaire).

3.1.2 Fonctions d'activation

Définition 3.2 (ReLU). La fonction *Rectified Linear Unit* : $\text{ReLU}(x) = \max(0, x)$. Elle introduit une non-linéarité tout en étant simple à calculer et à dériver ($\text{ReLU}'(x) = 1$ si $x > 0$, 0 sinon).

Définition 3.3 (Softmax). La fonction softmax transforme un vecteur $\mathbf{z} = (z_1, \dots, z_K) \in \mathbb{R}^K$ en une distribution de probabilité :

$$\text{softmax}(\mathbf{z})_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K. \quad (3.2)$$

On vérifie que $\text{softmax}(\mathbf{z})_k \geq 0$ et $\sum_k \text{softmax}(\mathbf{z})_k = 1$.

Remarque 3.1 (Pourquoi le softmax?). Dans notre contexte, l'agent RL doit choisir parmi K actions discrètes. Le réseau produit un vecteur de scores bruts $\mathbf{z} \in \mathbb{R}^K$ (*logits*). Le softmax convertit ces scores en probabilités, ce qui permet à l'agent d'échantillonner une action selon ces probabilités (exploration stochastique) ou de choisir l'action de plus grande probabilité (exploitation). Le choix du softmax plutôt que d'une autre normalisation est justifié par le fait qu'il est différentiable partout, qu'il respecte l'ordre des logits ($z_i > z_j \Rightarrow \text{softmax}(\mathbf{z})_i > \text{softmax}(\mathbf{z})_j$), et qu'il est le lien canonique de la distribution catégorielle en théorie des modèles exponentiels.

3.1.3 Réseau de neurones multicouche

Définition 3.4 (Réseau multicouche (*feedforward*)). Un réseau à L couches est la composition :

$$f_\theta(\mathbf{x}) = \sigma_L(W_L \cdot \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 \mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_{L-1}) + \mathbf{b}_L), \quad (3.3)$$

où $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ et $\mathbf{b}_\ell \in \mathbb{R}^{d_\ell}$ sont les paramètres de la couche ℓ , et $\theta = \{W_1, \mathbf{b}_1, \dots, W_L, \mathbf{b}_L\}$ l'ensemble de tous les paramètres.

L'apprentissage consiste à trouver θ minimisant une perte $L(\theta)$ par descente de gradient : $\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$, où $\eta > 0$ est le taux d'apprentissage. Le gradient est calculé par *rétropropagation* (application de la règle de dérivation en chaîne). L'optimiseur Adam, utilisé dans nos expériences, adapte le taux d'apprentissage pour chaque paramètre en utilisant les premier et second moments du gradient.

3.1.4 Mécanisme d’attention et Transformer

Le mécanisme d’attention, introduit par Vaswani et al. (2017) [?], est un composant central de l’architecture L2GLS. Son rôle est de permettre à chaque élément d’une séquence de “regarder” tous les autres éléments pour construire une représentation contextualisée.

Du TSP aux matrices Q, K, V . Dans notre contexte, la “séquence” est l’ensemble des N nœuds de la tournée, et chaque nœud est représenté par un vecteur de caractéristiques de dimension $d_{\text{in}} = 9 + 2H$ (coordonnées, distances, historique). L’entrée est donc la matrice $s \in \mathbb{R}^{N \times d_{\text{in}}}$, où chaque ligne est un nœud. Le mécanisme d’attention transforme cette matrice en une nouvelle matrice de même taille, où chaque ligne intègre l’information de *tous* les autres nœuds — pondérée par leur pertinence.

Comment mesurer la “pertinence” d’un nœud j pour un nœud i ? On utilise trois projections linéaires de s :

$$Q = sW^Q, \quad K = sW^K, \quad V = sW^V, \quad (3.4)$$

où $W^Q, W^K \in \mathbb{R}^{d_{\text{in}} \times d_k}$ et $W^V \in \mathbb{R}^{d_{\text{in}} \times d_v}$ sont des matrices de paramètres apprenables. Intuitivement :

- Q (*queries*) : chaque nœud exprime “ce qu’il cherche” — quelles informations lui seraient utiles ;
- K (*keys*) : chaque nœud exprime “ce qu’il offre” — quel type d’information il contient ;
- V (*values*) : le contenu effectif de chaque nœud.

Le score d’attention entre les nœuds i et j est le produit scalaire $q_i^\top k_j$: si la “requête” du nœud i correspond bien à la “clé” du nœud j , le score est élevé. Ces scores sont normalisés par softmax pour obtenir des poids d’attention.

Définition 3.5 (Attention par produit scalaire normalisé).

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \in \mathbb{R}^{N \times d_v}. \quad (3.5)$$

Le facteur $1/\sqrt{d_k}$ empêche les produits scalaires de devenir trop grands en dimension élevée, ce qui saturerait le softmax et produirait des gradients proches de zéro.

Définition 3.6 (Attention multi-têtes). L’attention multi-têtes utilise h têtes d’attention en parallèle, chacune avec ses propres projections, puis concatène et projette :

$$\text{MHA}(s) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attention}(sW_i^Q, sW_i^K, sW_i^V). \quad (3.6)$$

Cela permet au réseau de capturer différents types de relations (distance, voisinage dans la tournée, historique) simultanément.

Pourquoi l’attention pour le TSP ? Un réseau feedforward standard traite chaque nœud indépendamment. Or, la qualité d’un mouvement de recherche locale dépend des relations *entre* les nœuds (quelles arêtes sont longues, quels nœuds sont proches mais non adjacents dans la tournée). Le mécanisme d’attention capture ces relations en permettant à chaque nœud de “regarder” tous les autres, avec une pondération apprise.

3.2 Apprentissage par Renforcement : fondements

L’idée fondamentale du RL est la suivante : un *agent* interagit avec un *environnement* en prenant des *actions* et en recevant des *récompenses*. À travers cette interaction, l’agent apprend une *politique* — une règle de décision — qui maximise la récompense cumulée sur le long terme. Dans notre contexte, l’agent est le RL-Manager [1], l’environnement est la procédure GLS appliquée au TSP, les actions sont les choix d’opérateur et de pénalisation, et la récompense reflète l’amélioration de la tournée.

Le choix du RL plutôt que d’autres paradigmes d’apprentissage (apprentissage supervisé, *deep learning* pur) est justifié par la nature séquentielle du problème : l’agent prend une suite de décisions, chacune modifiant l’état de la recherche, et l’effet d’une décision ne se manifeste qu’après plusieurs étapes [1]. De plus, les données d’entraînement — des instances TSP — sont synthétiques et peuvent être générées en quantité arbitraire, ce qui est un avantage considérable.

Nous commençons par le formalisme général (le processus de décision de Markov), puis détaillons les trois algorithmes utilisés dans ce travail : REINFORCE, DQN et PPO. Pour chaque algorithme, nous donnons d’abord l’intuition, puis la formalisation, puis l’algorithme.

3.2.1 Processus de décision de Markov

Le formalisme standard de l’apprentissage par renforcement est le *processus de décision de Markov* (MDP). L’intuition est la suivante : à chaque instant t , l’agent observe un *état* s_t qui résume toute l’information pertinente sur la situation courante, choisit une *action* a_t , reçoit une *récompense* r_t , et le système passe à un nouvel état s_{t+1} . La propriété de Markov signifie que l’état futur ne dépend que de l’état courant et de l’action choisie, et non de l’historique complet.

Définition 3.7 (Processus de décision de Markov). Un MDP est un quintuplet $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ où :

1. $\mathcal{S}$ est l'*espace des états* : l'ensemble de toutes les situations possibles dans lesquelles l'agent peut se trouver ;
2. $\mathcal{A}$ est l'*espace des actions* : l'ensemble des décisions que l'agent peut prendre ;
3. $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ est la *fonction de transition* : $T(s' | s, a) = \Pr[s_{t+1} = s' | s_t = s, a_t = a]$ donne la probabilité de passer à l'état s' après avoir pris l'action a dans l'état s ;
4. $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ est la *fonction de récompense* : $R(s, a)$ est le signal que l'agent reçoit après avoir pris l'action a dans l'état s ;
5. $\gamma \in [0, 1]$ est le *facteur d'actualisation* : il pondère l'importance des récompenses futures par rapport aux récompenses immédiates. Une valeur γ proche de 1 favorise le long terme ; une valeur proche de 0 favorise le court terme.

L'agent agit selon une *politique*, qui est la règle par laquelle il choisit ses actions.

Définition 3.8 (Politique). Une politique $\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ associe à chaque état une distribution de probabilité sur les actions, paramétrée par un vecteur de paramètres θ :

$$\pi_\theta(a | s) = \Pr[a_t = a | s_t = s; \theta]. \quad (3.7)$$

Lorsque la politique est déterministe, elle associe à chaque état une unique action : $\pi(s) = a$.

Pour évaluer la qualité d'une politique, nous avons besoin de mesurer la récompense cumulée qu'elle génère.

Définition 3.9 (Fonction de valeur d'état). La fonction de valeur d'état $V^\pi(s)$ mesure la récompense cumulée espérée en partant de l'état s et en suivant la politique π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]. \quad (3.8)$$

Définition 3.10 (Fonction de valeur d'action). La fonction de valeur d'action $Q^\pi(s, a)$ mesure la récompense cumulée espérée en partant de l'état s , en prenant d'abord l'action a , puis en suivant la politique π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right]. \quad (3.9)$$

Définition 3.11 (Fonction avantage). La fonction avantage mesure la qualité relative d'une action par rapport à la moyenne :

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (3.10)$$

Si $A^\pi(s, a) > 0$, l'action a est meilleure que la moyenne des actions dans l'état s ; si $A^\pi(s, a) < 0$, elle est pire.

L'objectif de l'apprentissage est de trouver les paramètres θ^* maximisant la récompense cumulée espérée [11] :

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right]. \quad (3.11)$$

Deux grandes familles d'algorithmes existent pour résoudre ce problème : les méthodes de gradient de politique (qui optimisent directement π_θ) et les méthodes de valeur (qui apprennent Q ou V puis en déduisent la politique). Nous présentons un algorithme de chaque famille (REINFORCE et DQN), puis un algorithme hybride (PPO).

3.2.2 REINFORCE

REINFORCE, introduit par Williams (1992) [4], est l'algorithme de gradient de politique le plus simple. L'intuition est directe : si une action a conduit à une récompense élevée, on augmente la probabilité de choisir cette action dans des situations similaires à l'avenir ; si elle a conduit à une mauvaise récompense, on diminue cette probabilité.

Formellement, on cherche à maximiser $J(\theta)$ par montée de gradient. Le résultat fondamental suivant fournit une expression du gradient.

Théorème 3.1 (Théorème du gradient de politique). Le gradient de $J(\theta)$ admet l'estimateur sans biais :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=1}^M \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right], \quad (3.12)$$

où $G_t = \sum_{t'=t}^M R(s_{t'}, a_{t'})$ est le *retour cumulé* à partir du pas t : la somme de toutes les récompenses obtenues à partir de cet instant.

Le terme $\nabla_\theta \log \pi_\theta(a_t | s_t)$ indique la direction dans laquelle modifier θ pour augmenter la probabilité de l'action a_t . Le retour cumulé G_t joue le rôle de pondération : si G_t est positif (bonne trajectoire), on augmente la probabilité de a_t ; s'il est négatif, on la diminue.

En pratique, la variance de cet estimateur est élevée, car G_t fluctue beaucoup d'un épisode à l'autre. Pour réduire cette variance sans introduire de biais, on soustrait une *baseline* $b(s_t)$ (typiquement une moyenne des retours) :

$$\nabla_\theta J(\theta) \approx \sum_{t=1}^M (G_t - b(s_t)) \cdot \nabla_\theta \log \pi_\theta(a_t | s_t). \quad (3.13)$$

Algorithm 3: REINFORCE avec baseline

Entrée: Politique initiale π_θ , taux d'apprentissage η , horizon M

Sortie: Politique optimisée π_{θ^*}

```
1 for chaque épisode do
2   Collecter la trajectoire  $\tau = \{(s_t, a_t, r_t)\}_{t=1}^M$  en suivant  $\pi_\theta$ ;
3   for  $t \leftarrow 1$  to  $M$  do
4      $G_t \leftarrow \sum_{t'=t}^M r_{t'}$  ; // Retour cumulé
5      $\delta_t \leftarrow G_t - b(s_t)$  ; // Avantage estimé
6   end
7    $\theta \leftarrow \theta + \eta \cdot \nabla_\theta \sum_{t=1}^M \delta_t \cdot \log \pi_\theta(a_t | s_t)$ ;
8 end
9 return  $\pi_\theta$ 
```

REINFORCE est simple et optimise directement la politique. Sa principale faiblesse est la variance élevée de l'estimateur du gradient, qui peut ralentir la convergence pour des épisodes longs (grand M), ce qui est précisément le cas dans L2GLS ($M = 40\,000$).

3.2.3 Deep Q-Network (DQN)

Le DQN, introduit par Mnih et al. (2015) [?], adopte une approche radicalement différente de REINFORCE. Au lieu de paramétrer directement la politique, le DQN apprend la *fonction de valeur d'action* $Q(s, a)$ — c'est-à-dire la récompense cumulée espérée associée à chaque couple (état, action) — puis en déduit la politique de manière implicite en choisissant toujours l'action de plus grande valeur : $\pi(s) = \arg \max_a Q(s, a)$.

L'intuition est la suivante : si l'on connaissait parfaitement la valeur de chaque action dans chaque état, le choix optimal serait trivial — il suffirait de toujours choisir l'action de plus grande valeur. Le DQN utilise un réseau de neurones pour *approximer* cette fonction de valeur à partir de l'expérience.

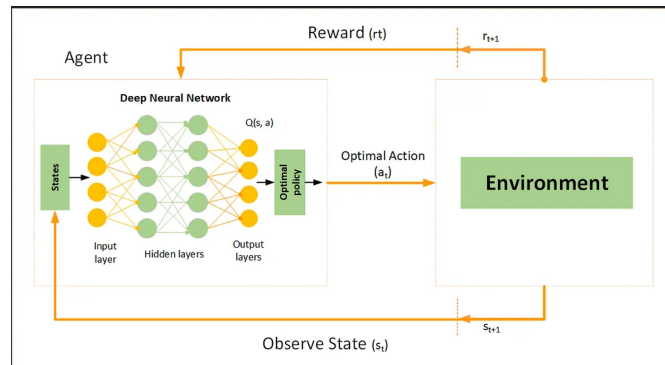


FIGURE 3.1 – Architecture du Deep Q-Network (DQN) : le réseau approxime la fonction $Q(s, a)$ pour chaque action possible à partir de l'état courant.

Définition 3.12 (Fonction de valeur d'action optimale). La fonction Q^* satisfait l'équation de Bellman optimale : la valeur optimale d'une action a dans un état s est la récompense immédiate plus la meilleure valeur atteignable depuis le prochain état :

$$Q^*(s, a) = \mathbb{E}_{s'} \left[R(s, a) + \gamma \cdot \max_{a'} Q^*(s', a') \right]. \quad (3.14)$$

Le DQN apprend Q^* en minimisant l'écart entre la prédiction du réseau $Q_\theta(s, a)$ et la cible de Bellman. Deux mécanismes assurent la stabilité de cet apprentissage :

L'*Experience Replay* consiste à stocker les transitions (s, a, r, s') dans un tampon mémoire $\mathcal{D}$ de taille $|\mathcal{D}|_{\max}$, et à entraîner le réseau sur des mini-batches échantillonnés aléatoirement dans $\mathcal{D}$. Ce mécanisme brise les corrélations temporelles entre transitions successives, qui déstabiliseraient l'apprentissage.

Le *Target Network* consiste à utiliser un second réseau Q_{θ^-} (appelé réseau cible) pour calculer les valeurs cibles dans l'équation de Bellman. Ses paramètres θ^- sont copiés depuis θ tous les C pas, ce qui stabilise la cible d'apprentissage.

La perte à minimiser est :

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y - Q_\theta(s, a))^2 \right], \quad y = r + \gamma \cdot \max_{a'} Q_{\theta^-}(s', a'). \quad (3.15)$$

Algorithm 4: Deep Q-Network (DQN)

Entrée: Réseau Q_θ , réseau cible Q_{θ^-} , $|\mathcal{D}|_{\max}$, $|B|$, C , γ , ε

Sortie: Réseau Q_{θ^*} entraîné

```

1 Initialiser  $\theta$ ;  $\theta^- \leftarrow \theta$ ;  $\mathcal{D} \leftarrow \emptyset$ ;
2 for chaque épisode do
3   for  $t \leftarrow 1$  to  $M$  do
4     Avec proba  $\varepsilon$  :  $a_t \sim \text{Uniform}(\mathcal{A})$ , sinon  $a_t = \arg \max_a Q_\theta(s_t, a)$ ;
5     Exécuter  $a_t$ , observer  $r_t, s_{t+1}$ ;
6     Stocker  $(s_t, a_t, r_t, s_{t+1})$  dans  $\mathcal{D}$ ;
7     if  $|\mathcal{D}| \geq |B|$  then
8       Mini-batch  $B \sim \mathcal{D}$ ;
9        $y^{(j)} \leftarrow r^{(j)} + \gamma \cdot \max_{a'} Q_{\theta^-}(s'^{(j)}, a')$  pour chaque  $(s^{(j)}, a^{(j)}, r^{(j)}, s'^{(j)}) \in B$ ;
10       $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \frac{1}{|B|} \sum_j (y^{(j)} - Q_\theta(s^{(j)}, a^{(j)}))^2$ ;
11    end
12    if  $t \bmod C = 0$  then  $\theta^- \leftarrow \theta$ ;
13  end
14 end
15 return  $Q_\theta$ 

```

Le DQN offre une estimation plus stable des rendements futurs et réutilise efficacement

les données. En revanche, il est limité aux espaces d’actions discrets et peut souffrir de surestimation des Q -valeurs, phénomène partiellement atténué par le réseau cible.

3.2.4 Proximal Policy Optimization (PPO)

Le PPO, introduit par Schulman et al. (2017) [5], appartient à la famille des algorithmes *actor-critic* : il combine un *acteur* (qui paramètre la politique π_θ) et un *critique* (qui estime la fonction de valeur V_ϕ). L’intuition est de prendre le meilleur des deux mondes : l’optimisation directe de la politique (comme REINFORCE) avec la réduction de variance permise par un critique (comme dans le DQN).

Le problème avec REINFORCE est que le gradient peut être très variable d’un épisode à l’autre [5]. Le problème avec une mise à jour trop agressive de la politique est qu’elle peut détruire ce que l’agent a appris [5]. Le PPO résout ce problème en limitant l’amplitude des mises à jour par un mécanisme de *clipping* [5] : on autorise la politique à changer, mais pas “trop” d’un coup.

Définition 3.13 (Ratio de probabilité). Le ratio entre la politique courante et l’ancienne politique est :

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (3.16)$$

Si $r_t(\theta) > 1$, la politique courante est plus susceptible de choisir l’action a_t que l’ancienne. Si $r_t(\theta) < 1$, elle est moins susceptible.

Définition 3.14 (Objectif PPO-Clip). L’objectif clippé est :

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right], \quad (3.17)$$

où $\epsilon > 0$ est le paramètre de clipping (typiquement $\epsilon = 0,2$) et $\hat{A}_t$ est l’estimateur de l’avantage.

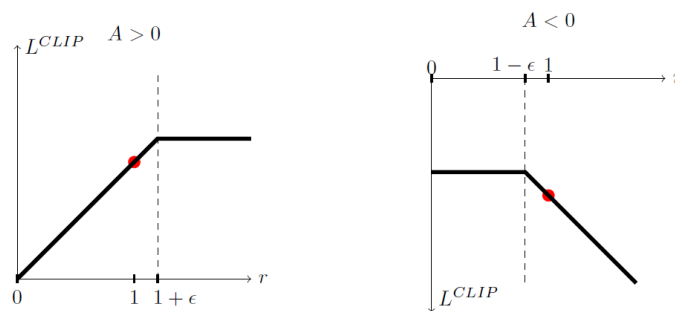


FIGURE 3.2 – Illustration de l’objectif clippé L^{CLIP} : le clipping empêche des mises à jour trop importantes de la politique selon le signe de l’avantage $\hat{A}_t$.

L'intuition de cette formule est la suivante. Lorsque l'avantage est positif ($\hat{A}_t > 0$), on souhaite augmenter la probabilité de l'action, ce qui augmente $r_t(\theta)$. Mais le min avec le terme clippé empêche r_t de dépasser $1 + \epsilon$: la politique ne peut pas changer trop brutalement. Symétriquement, lorsque $\hat{A}_t < 0$, le clipping empêche r_t de descendre en dessous de $1 - \epsilon$.

L'estimateur de l'avantage utilisé est le GAE (*Generalized Advantage Estimation*) [?], qui offre un compromis contrôlable entre biais et variance.

Définition 3.15 (GAE). L'estimateur GAE [5] est :

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda_{\text{GAE}})} = \sum_{l=0}^{T-t} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l}, \quad (3.18)$$

où $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ est l'*erreur TD résiduelle* (la différence entre la récompense observée et la prédiction du critique), et $\lambda_{\text{GAE}} \in [0, 1]$ contrôle le compromis biais-variance : $\lambda_{\text{GAE}} = 0$ donne un estimateur à faible variance mais biaisé, $\lambda_{\text{GAE}} = 1$ donne un estimateur sans biais mais à haute variance.

La perte totale du PPO combine trois termes :

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_1 \cdot L^{\text{VF}}(\phi) - c_2 \cdot H[\pi_\theta], \quad (3.19)$$

où $L^{\text{VF}}(\phi) = \mathbb{E}_t[(V_\phi(s_t) - G_t)^2]$ est la perte du critique (qui apprend à prédire les retours cumulés), $H[\pi_\theta] = -\sum_a \pi_\theta(a | s) \log \pi_\theta(a | s)$ est l'entropie de la politique (un bonus qui encourage l'exploration en pénalisant les politiques trop déterministes), et c_1, c_2 sont des coefficients de pondération.

Algorithm 5: Proximal Policy Optimization (PPO-Clip)

Entrée: Politique π_θ , critique V_ϕ , paramètres $\epsilon, \gamma, \lambda_{\text{GAE}}, K$

Sortie: Politique optimisée π_{θ^*}

```
1 for chaque itération do
2   Collecter  $T$  pas avec  $\pi_{\theta_{\text{old}}}$ ;
3   Calculer  $\hat{A}_t^{\text{GAE}}$  pour chaque pas  $t$ ;
4    $\theta_{\text{old}} \leftarrow \theta$ ;
5   for  $k \leftarrow 1$  to  $K$  do
6     for chaque mini-batch  $B$  do
7        $r_t(\theta) \leftarrow \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ ;
8        $L^{\text{CLIP}} \leftarrow \frac{1}{|B|} \sum_{t \in B} \min(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)$ ;
9        $L^{\text{VF}} \leftarrow \frac{1}{|B|} \sum_{t \in B} (V_\phi(s_t) - G_t)^2$ ;
10       $L \leftarrow -L^{\text{CLIP}} + c_1 L^{\text{VF}} - c_2 H[\pi_\theta]$ ;
11       $(\theta, \phi) \leftarrow (\theta, \phi) - \eta \cdot \nabla L$ ;
12    end
13  end
14 end
15 return  $\pi_\theta$ 
```

3.2.5 Synthèse comparative

REINFORCE optimise directement la politique mais souffre d'une variance élevée. Le DQN apprend une fonction de valeur avec plus de stabilité, mais est limité aux actions discrètes et peut surestimer les Q -valeurs. Le PPO combine les avantages des deux : il optimise directement la politique tout en utilisant un critique pour réduire la variance, avec un mécanisme de clipping qui garantit des mises à jour stables. De plus, sa nature *on-policy* le rend adapté aux environnements non stationnaires — ce qui est notre cas, puisque le paysage de la fonction h évolue à chaque mise à jour de pénalité.

Le chapitre suivant formalise l'application de ces trois algorithmes au cadre L2GLS pour le TSP.

Chapitre 4

L2GLS — Formalisation et Contributions

Ce chapitre constitue le cœur de notre travail. Les chapitres précédents ont établi les trois piliers nécessaires : le TSP (chapitre 1), les opérateurs de recherche locale et la GLS (chapitre 2), et les algorithmes d'apprentissage par renforcement (chapitre 3). Nous sommes maintenant en mesure de formaliser la méthode L2GLS : comment modéliser le pilotage de la GLS comme un problème de décision séquentielle, et comment le résoudre avec REINFORCE, DQN et PPO.

4.1 Architecture générale

L'architecture de L2GLS couple deux niveaux de décision. Au niveau inférieur, la GLS avec recherche locale opère directement sur la tournée : elle applique un opérateur, évalue le voisinage, pénalise des features. Au niveau supérieur, un agent RL (le RL-Manager) observe l'état courant de la recherche et décide : quel opérateur appliquer, et combien de pénalisations GLS effectuer avant de reprendre la recherche locale.

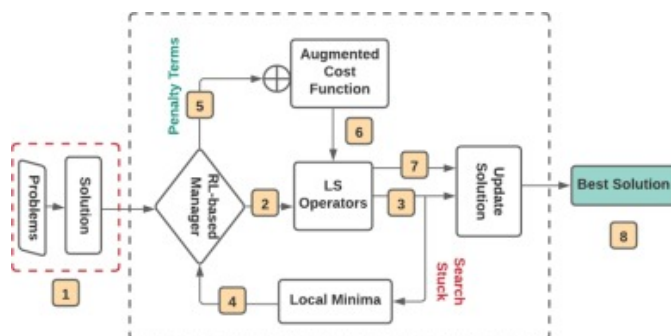


FIGURE 4.1 – Architecture générale de L2GLS : couplage entre le RL-Manager (niveau supérieur) et la procédure GLS avec recherche locale (niveau inférieur).

Concrètement, à chaque pas de temps t , le RL-Manager : (1) observe l'état s_t de la tournée courante ; (2) choisit une action a_t (un couple opérateur/pénalisations) ; (3) l'environnement applique l'action (recherche locale puis pénalisations GLS) et passe au nouvel état s_{t+1} ; (4) l'agent reçoit une récompense r_t indiquant si la tournée s'est améliorée.

Ce couplage est formalisé par un MDP dont nous définissons maintenant chaque composante.

4.2 Formulation MDP pour le TSP

4.2.1 Espace d'états $\mathcal{S}$

L'état doit contenir toute l'information nécessaire au RL-Manager pour prendre une bonne décision. L'intuition est la suivante : l'agent a besoin de connaître la structure locale de la tournée autour de chaque nœud (pour estimer le potentiel d'amélioration), ainsi que l'historique récent des actions et de leurs effets (pour éviter de répéter des actions inefficaces).

Pour chaque nœud i de la tournée courante π , le vecteur de caractéristiques comprend : les coordonnées du nœud $(m_i, n_i) \in \mathbb{R}^2$, les coordonnées du prédécesseur (m_i^-, n_i^-) et du successeur (m_i^+, n_i^+) dans la tournée, les distances $d(i^-, i)$, $d(i, i^+)$ et $d(i^-, i^+)$, et enfin les H dernières actions prises et leurs effets (+1 si la tournée s'est améliorée, -1 sinon).

Formellement, l'état courant est une matrice :

$$s \in \mathbb{R}^{N \times (9+2H)}, \quad (4.1)$$

dont la ligne i est :

$$s_i = (m_i, n_i, m_i^-, n_i^-, m_i^+, n_i^+, d(i^-, i), d(i, i^+), d(i^-, i^+), a_{t-1}, r_{t-1}, \dots, a_{t-H}, r_{t-H}). \quad (4.2)$$

Les 9 premières composantes capturent la géométrie locale de la tournée ; les $2H$ dernières capturent la dynamique récente de la recherche. Cette représentation est partagée entre les trois algorithmes (REINFORCE, DQN, PPO).

4.2.2 Espace d'actions $\mathcal{A}$

La décision du RL-Manager comporte deux volets : quel opérateur de recherche locale appliquer, et combien de pénalisations GLS effectuer. Deux formulations de l'espace d'actions sont utilisées dans ce travail.

Espace plat (REINFORCE et DQN)

Définition 4.1 (Action — espace plat). Chaque action est un couple $a = (o, \varphi)$ où $o \in \mathcal{O} = \{2\text{-opt}, \text{relocate}, \text{swap}, \text{three-permutation}\}$ est l’opérateur, et $\varphi \in \Phi = \{1, 2, 3\}$ est le nombre de pénalisations GLS. L’espace d’actions contient $|\mathcal{A}| = 4 \times 3 = 12$ actions discrètes.

Espace hiérarchique (PPO)

Définition 4.2 (Action — espace hiérarchique). L’espace d’actions est décomposé en deux niveaux séquentiels : (1) choix du nombre de pénalités $\varphi \in \{0, 1, 3\}$ (incluant $\varphi = 0$ pour aucune pénalisation) ; (2) choix de l’opérateur $o \in \{2\text{-opt}, \text{relocate}, \text{swap}, \text{three-permutation}\}$.

Cette décomposition hiérarchique reflète la structure naturelle de la décision dans la GLS : on détermine d’abord le niveau de diversification (combien pénaliser), puis l’opérateur d’intensification (comment améliorer). L’inclusion de $\varphi = 0$ permet à l’agent de choisir de ne pas pénaliser, ce qui est utile lorsque la tournée courante est dans une bonne région.

4.2.3 Fonction de transition T

La transition est déterministe. Étant donné l’état s et l’action $a = (o, \varphi)$:

$$T(s, a) = \text{StateOf}\left(\text{GLSUpdate}_{\varphi}(\text{LocalSearch}_I(s, o, h))\right), \quad (4.3)$$

où LocalSearch_I désigne $I = 6$ étapes d’amélioration avec l’opérateur o sur la fonction augmentée h (éq. (2.15)), et $\text{GLSUpdate}_{\varphi}$ désigne φ pénalisations GLS sur la feature d’utilité maximale (éq. (2.16)). L’enchaînement est : d’abord on améliore la tournée avec l’opérateur, puis on met à jour les pénalités GLS.

4.2.4 Signal de récompense R

Le signal de récompense indique au RL-Manager si son action a été bénéfique. L’article original propose deux variantes.

Récompense binaire (FR). Signal simple indiquant si la meilleure tournée connue a été améliorée :

$$R_{\text{FR}}(s_t, a_t) = \begin{cases} +1 & \text{si } g(\pi_{t+1}) < g(\pi_t^*), \\ -1 & \text{sinon.} \end{cases} \quad (4.4)$$

Récompense continue (SR). Signal nuancé mesurant l’amplitude de l’amélioration, avec un facteur de décroissance temporelle $\rho \in (0, 1]$ encourageant les améliorations précoces :

$$R_{\text{SR}}(s_t, a_t) = \frac{g(\pi_t^*) - g(\pi_{t+1}^*)}{g(\pi_t^*)} \cdot \rho^t. \quad (4.5)$$

Récompense normalisée (contribution PPO). Pour le PPO, nous utilisons une récompense normalisée par le coût initial :

$$R_{\text{PPO}}(s_t, a_t) = \frac{g(\pi_t) - g(\pi_{t+1})}{g(\pi_0)}. \quad (4.6)$$

Cette normalisation assure que la magnitude de la récompense est comparable quelle que soit la taille de l’instance. Cette propriété est importante pour l’entraînement par curriculum (section 4.5.3), où l’agent est entraîné simultanément sur des instances de tailles variées.

4.3 L2GLS avec REINFORCE (méthode originale)

4.3.1 Architecture du réseau

Le réseau de politique prend en entrée la matrice d’état $s \in \mathbb{R}^{N \times (9+2H)}$ et produit une distribution de probabilité sur les 12 actions. L’architecture comprend quatre étapes.

Premièrement, une couche d’attention multi-têtes (*Multi-Head Attention*, MHA) calcule, pour chaque nœud, une représentation contextualisée qui prend en compte les autres nœuds :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V, \quad (4.7)$$

où Q, K, V sont des projections linéaires de s et d_k est la dimension des clés. Le facteur $1/\sqrt{d_k}$ normalise les produits scalaires pour éviter que le softmax ne sature.

Deuxièmement, un *pooling* global (moyenne sur les N nœuds) agrège les représentations en un vecteur de taille fixe : $z = \frac{1}{N} \sum_{i=1}^N \text{MHA}(s)_i \in \mathbb{R}^{d_{\text{model}}}$.

Troisièmement, deux couches entièrement connectées avec activation ReLU : $h_1 = \text{ReLU}(W_1 z + b_1)$, $h_2 = \text{ReLU}(W_2 h_1 + b_2)$.

Quatrièmement, une couche de sortie softmax sur $|\mathcal{A}| = 12$ actions : $\pi_\theta(\cdot \mid s) = \text{softmax}(W_3 h_2 + b_3)$.

4.3.2 Procédure d'entraînement

L'entraînement suit le protocole de l'article original : instances avec coordonnées dans $[0, 1]^2$ pour $N \in \{20, 50, 100\}$, $M = 40\,000$ pas de rollout, $I = 6$ étapes d'amélioration par action, $\lambda = 0,3 \cdot g(\pi_{nn}^*)/N$, exploration ε -greedy avec $\varepsilon = 0,05$, optimiseur Adam avec $\eta = 10^{-3}$.

4.3.3 Algorithme complet L2GLS (inférence)

Algorithm 6: L2GLS — Inférence

Entrée: Instance TSP (N villes), politique π_θ , paramètres $M, I, \lambda, \varepsilon$

Sortie: Meilleure tournée π^*

```

1  $\pi_0 \leftarrow$  plus proche voisin ;  $P \leftarrow 0$  ;  $\pi^* \leftarrow \pi_0$ ;
2  $\lambda \leftarrow 0,3 \cdot g(\pi_{nn}^*)/N$ ;
3 for  $t = 1$  to  $M$  do
4   Construire  $s_t \in \mathbb{R}^{N \times (9+2H)}$ ;
5   Avec proba  $\varepsilon$  :  $a_t \sim \text{Uniform}(\mathcal{A})$ , sinon  $a_t = \arg \max_a \pi_\theta(a \mid s_t)$ ;
6    $(o_t, \varphi_t) \leftarrow a_t$ ;
7   for  $\ell = 1$  to  $I$  do
8     Appliquer un mouvement améliorant de  $o_t$  sur  $h = g + \lambda \langle P, I_e \rangle$ ;
9   end
10  for  $k = 1$  to  $\varphi_t$  do
11     $e^* \leftarrow \arg \max_{e_{ij} \in \pi_t} I_{e_{ij}}(\pi_t) \cdot d_{ij} / (1 + p_{ij})$ ;
12     $p_{e^*} \leftarrow p_{e^*} + 1$ ;
13  end
14  if  $g(\pi_t) < g(\pi^*)$  then  $\pi^* \leftarrow \pi_t$ ;
15 end
16 return  $\pi^*$ 

```

4.4 Contribution 1 : L2GLS avec DQN

Notre première contribution remplace REINFORCE par un DQN. Le MDP reste identique ; seule la méthode d'apprentissage change. Le réseau Q_θ reprend la même architecture d'entrée (MHA, pooling) mais la couche de sortie produit 12 valeurs Q au lieu d'une distribution de probabilité :

$$Q_\theta(s, \cdot) = W_3 h_2 + b_3 \in \mathbb{R}^{12}. \quad (4.8)$$

L'action est sélectionnée par $\arg \max$ avec exploration ε -greedy. Les hyperparamètres

du DQN sont : $|\mathcal{D}|_{\max} = 10^4$, $|B| = 64$, $C = 1\,000$, $\gamma = 0,99$.

L’objectif est d’évaluer si l’estimation de la fonction de valeur et la réutilisation des données via l’*experience replay* compensent la perte de l’optimisation directe de la politique.

4.5 Contribution 2 : L2GLS avec PPO et espace hiérarchique

Notre seconde contribution introduit deux modifications : l’algorithme PPO et un espace d’actions hiérarchique à deux niveaux (définition 4.2).

4.5.1 Justification du PPO pour L2GLS

Le PPO est motivé par trois considérations propres au problème L2GLS. Premièrement, la *non-stationnarité* : le paysage de la fonction h évolue à chaque pénalisation GLS ; le clipping (équ. (3.17)) assure des mises à jour conservatrices adaptées. Deuxièmement, la *réduction de variance* : le critique V_ϕ et le GAE (équ. (3.18)) réduisent la variance par rapport à REINFORCE, ce qui est critique pour $M = 40\,000$ pas. Troisièmement, l’*efficacité d’échantillonnage* : les K passes par rollout réutilisent les données collectées.

4.5.2 Architecture du réseau

Le réseau possède un tronc commun suivi de trois têtes : la tête de pénalité produit $\text{softmax}(W_p h_{\text{trunk}} + b_p) \in \Delta^2$ (3 logits pour $\varphi \in \{0, 1, 3\}$) ; la tête d’opérateur produit $\text{softmax}(W_o h_{\text{trunk}} + b_o) \in \Delta^3$ (4 logits pour les opérateurs) ; la tête de valeur produit $V_\phi(s) = W_v h_{\text{trunk}} + b_v \in \mathbb{R}$.

La log-probabilité de l’action jointe est la somme des log-probabilités des deux sous-décisions :

$$\log \pi_\theta(a \mid s) = \log \pi_\theta^{\text{pen}}(\varphi \mid s) + \log \pi_\theta^{\text{op}}(o \mid s). \quad (4.9)$$

4.5.3 Entraînement par curriculum

L’entraînement utilise un curriculum par mélange de tailles d’instances : à chaque épisode, la taille N est échantillonnée dans $\{20, 50, 100\}$. Cette approche par batches mixtes est préférable à un curriculum séquentiel car la nature *on-policy* du PPO rend problématique le changement de distribution entre phases. La récompense normalisée (équ. (4.6)) assure la compatibilité des magnitudes entre tailles.

4.5.4 Hyperparamètres

Les hyperparamètres sont calibrés pour un GPU GTX 1650 (4 Go VRAM) : dimension cachée $d = 64$, batch $|B| = 32$, $K = 4$ passes, $\epsilon = 0,2$, $\gamma = 0,99$, $\lambda_{\text{GAE}} = 0,95$, $c_1 = 0,5$, $c_2 = 0,01$. Le diagnostic principal de l'entraînement est la divergence KL approximative entre l'ancienne et la nouvelle politique, dont la valeur saine se situe dans $[0,005, 0,015]$.

4.6 Implémentation logicielle

L'intégralité du code a été développée *from scratch* en Python 3.12, sans s'appuyer sur des frameworks de RL existants (tels que Stable-Baselines3 ou RLlib). Ce choix est motivé par deux raisons : d'une part, la maîtrise complète de chaque composant permet de comprendre et de déboguer finement les interactions entre la GLS et l'agent RL ; d'autre part, l'environnement L2GLS, qui couple recherche locale et pénalisation GLS, ne correspond à aucune interface standard (type Gymnasium) et nécessite une implémentation sur mesure. Le code est structuré en 9 modules Python, chacun ayant une responsabilité unique :

1. `tsp_problem.py` : représentation d'une instance TSP (coordonnées, matrice de distances) et heuristique du plus proche voisin ;
2. `operators.py` : les cinq opérateurs de recherche locale (2-opt, 3-opt, relocate, swap, three-permutation), chacun acceptant une matrice de distances augmentée pour la GLS ;
3. `gls_penalties.py` : le mécanisme de pénalisation GLS (compteurs par arête, calcul de l'utilité, mise à jour des distances augmentées) ;
4. `state_representation.py` : construction du vecteur d'état 11-dimensionnel normalisé pour l'agent RL ;
5. `environment.py` : l'environnement L2GLS au format Gym-like (méthodes `reset`, `step`), qui orchestre l'enchaînement pénalisation–opérateur–récompense ;
6. `agents.py` : les trois agents (REINFORCE, DQN, PPO) partageant une interface commune (`select_action`, `update`, `save`, `load`) ;
7. `config.py` : hyperparamètres centralisés pour chaque agent ;
8. `training.py` : boucle d'entraînement générique, adaptée automatiquement au type d'agent (per-step pour DQN, per-episode pour REINFORCE, per-rollout pour PPO) ;
9. `testing.py` : baselines classiques (NN, 2-opt, 3-opt, GLS+2-opt, GLS+3-opt) et benchmark comparatif.

Le framework de deep learning utilisé est PyTorch 2.x. Les réseaux de neurones des trois agents utilisent des couches entièrement connectées (*feedforward*) avec activation ReLU,

sans recourir au mécanisme d’attention multi-têtes décrit dans le papier original [1]. Ce choix simplifié est dicté par les contraintes matérielles (4 Go VRAM) : un réseau feedforward à 2 couches cachées de 64 neurones consomme environ 50 ko de paramètres, contre plusieurs Mo pour un Transformer, ce qui permet un entraînement rapide sur GPU grand public. L’architecture feedforward constitue une limitation : elle traite l’état comme un vecteur global agrégé (11 dimensions) plutôt que comme une matrice par noeud, ce qui perd l’information structurelle locale. L’utilisation d’un Transformer ou d’un *Graph Neural Network* constitue une perspective d’amélioration directe.

L’ensemble du code est disponible sous forme d’un pipeline unique : la commande `python run_all.py` lance l’entraînement des trois agents puis exécute les benchmarks comparatifs sur instances aléatoires et TSPLIB, en sauvegardant tous les résultats au format CSV et JSON.

Chapitre 5

Protocole expérimental et résultats

Ce chapitre présente le protocole expérimental et analyse les résultats obtenus. L’objectif est double : comparer les trois implémentations de L2GLS (REINFORCE, DQN, PPO) entre elles, et les positionner par rapport aux méthodes de référence classiques et aux résultats publiés dans la littérature — notamment Concorde [?] (solveur exact, solution optimale), L2GLS original [1] et L2I [9].

5.1 Configuration matérielle et logicielle

Tous les entraînements et benchmarks sont réalisés sur un ordinateur portable ASUS TUF Gaming F15 équipé d’un processeur Intel Core i5-10300H (4 cœurs, 2,5 GHz), d’une carte graphique NVIDIA GeForce GTX 1650 (4 Go VRAM) et de 8 Go de RAM. L’implémentation utilise PyTorch 2.x et le code est structuré en 9 modules Python. Ce choix matériel modeste est délibéré : il valide que L2GLS peut être entraîné et déployé sans ressources de calcul dédiées, contrairement à de nombreuses approches de la littérature qui requièrent des GPU de type V100 ou A100.

5.2 Configuration d’entraînement

Les trois agents sont entraînés sur des instances aléatoires dont les coordonnées sont uniformément distribuées dans $[0, 1]^2$, avec des tailles $N \in \{20, 50, 100\}$ échantillonnées aléatoirement à chaque épisode (curriculum par mélange). Les paramètres communs sont : $\lambda = 0,3 \cdot g(\pi_{nn}^*)/N$, $I = 6$ étapes d’amélioration par action, 200 pas de décision par épisode, optimiseur Adam, et exploration ε -greedy avec $\varepsilon = 0,05$.

Le nombre d’épisodes d’entraînement est adapté à chaque algorithme en fonction de son efficacité d’échantillonnage, comme le résume le tableau 5.1.

TABLE 5.1 – Configuration d’entraînement par agent. Le nombre d’épisodes est adapté à l’efficacité d’échantillonnage de chaque algorithme.

Agent	Épisodes	LR	γ	Spécifique	Temps (min)
REINFORCE	1 500	5×10^{-4}	0,99	baseline $\alpha = 0,01$	21,1
DQN	1 000	10^{-3}	0,99	$ \mathcal{D} = 20\text{k}$, $C = 100$	24,2
PPO	800	3×10^{-4}	0,99	$\epsilon=0,2$, $K=4$, $\lambda_{\text{GAE}}=0,95$	22,6

REINFORCE nécessite le plus grand nombre d’épisodes (1 500) car il est *on-policy* et souffre d’une variance élevée : chaque épisode ne contribue qu’une seule mise à jour du gradient. Le DQN, grâce à l’*experience replay*, réutilise chaque transition plusieurs fois et converge en 1 000 épisodes. Le PPO est le plus efficace en échantillonnage grâce aux $K = 4$ passes par rollout, d’où les 800 épisodes. Les trois agents sont entraînés dans un temps comparable (~ 21 – 24 minutes), comme l’illustre la figure 5.1.

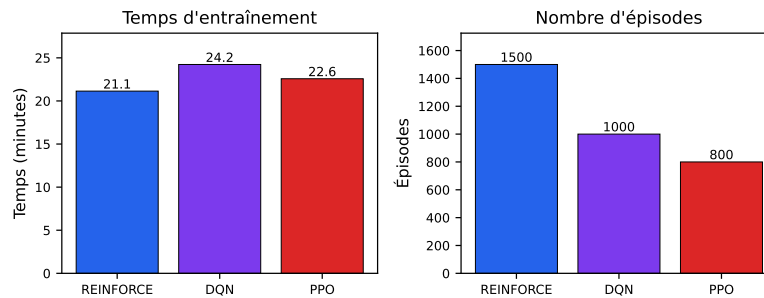


FIGURE 5.1 – Temps d’entraînement (en minutes) et nombre d’épisodes par agent. Malgré des nombres d’épisodes différents, les temps totaux sont comparables grâce à la différence de coût par épisode.

La figure 5.2 présente les courbes de convergence des trois agents. REINFORCE et DQN convergent de manière stable vers une amélioration de 25–30% par rapport à la solution initiale (plus proche voisin). Le PPO montre une convergence plus lente et une variance plus élevée, ce que nous analyserons en section 5.8.



FIGURE 5.2 – Courbes de convergence : amélioration (%) par épisode avec moyenne glissante (fenêtre de 30 épisodes). Les courbes brutes (en transparence) montrent la variance épisodique ; la ligne pleine montre la tendance.

5.3 Instances de test

Les tests sont effectués sur deux types d’instances pour mesurer à la fois la performance moyenne et la capacité de généralisation.

Les *instances aléatoires* sont générées pour chaque taille $N \in \{20, 50, 100, 200\}$: 20 instances par taille avec coordonnées dans $[0, 1]^2$, en utilisant des graines de génération différentes de celles de l’entraînement. Pour ces instances, la solution de référence est obtenue par la meilleure méthode testée (en pratique, REINFORCE).

Les *instances TSPLIB* [?] sont des benchmarks classiques dont les solutions optimales, vérifiées par le solveur exact Concorde [?], sont connues. Nous utilisons 32 instances EUC_2D de taille $n = 51$ à $n = 493$. Le gap d’optimalité est calculé par rapport à ces solutions exactes, ce qui permet une comparaison directe et non ambiguë avec toutes les méthodes, y compris celles de la littérature.

5.4 Métriques d’évaluation

Le *gap d’optimalité* mesure l’écart relatif entre la solution trouvée et la solution optimale de Concorde :

$$\text{Gap}(\%) = 100 \times \frac{g(\pi) - g(\pi_{\text{Concorde}}^*)}{g(\pi_{\text{Concorde}}^*)}. \quad (5.1)$$

Un gap de 0% signifie que la méthode a trouvé la solution optimale exacte. Le *temps d’inférence* est le temps moyen en secondes pour résoudre une instance *après* entraînement ; il ne comprend pas le temps d’entraînement, qui est un coût fixe amorti.

5.5 Méthodes comparées

Nous comparons huit méthodes, classées de la plus simple à la plus complexe : (1) plus proche voisin (NN) ; (2) recherche locale pure avec 2-opt ; (3) recherche locale pure avec 3-opt ; (4) GLS + 2-opt ; (5) GLS + 3-opt ; (6) L2GLS (REINFORCE) ; (7) L2GLS (DQN) ; (8) L2GLS (PPO). Pour les instances TSPLIB, nous ajoutons trois références de la littérature : Concorde [?] (gap = 0%, optimal exact), les résultats publiés de L2GLS [1] et ceux de L2I [9], sur les instances pour lesquelles ces données sont disponibles.

5.6 Résultats sur instances aléatoires

Le tableau 5.2 présente les résultats moyens sur les 20 instances aléatoires par taille. Le gap est calculé par rapport à la meilleure méthode pour chaque taille.

TABLE 5.2 – Résultats sur instances aléatoires (20 instances par taille). Gap (%) relatif à la meilleure méthode par taille. La meilleure méthode est en **gras**.

Méthode	Gap moyen (%)				Temps moyen (s)			
	$N=20$	$N=50$	$N=100$	$N=200$	$N=20$	$N=50$	$N=100$	$N=200$
NN	15,82	24,73	24,65	22,00	<0,01	<0,01	<0,01	<0,01
2-opt	0,98	3,30	3,23	2,19	<0,01	0,01	0,05	0,43
3-opt	1,17	2,84	2,02	2,42	0,01	0,34	4,55	7,46
GLS+2-opt	0,22	1,88	2,23	1,61	0,03	0,14	0,51	2,30
GLS+3-opt	0,15	1,91	1,66	1,91	0,19	3,42	29,12	101,5
L2GLS (DQN)	0,02	0,03	0,13	0,06	0,98	1,85	6,17	16,76
L2GLS (RF)	0,00	0,00	0,00	0,00	0,84	1,54	5,08	17,58
L2GLS (PPO)	2,76	5,85	6,71	5,97	1,24	2,50	7,46	25,49

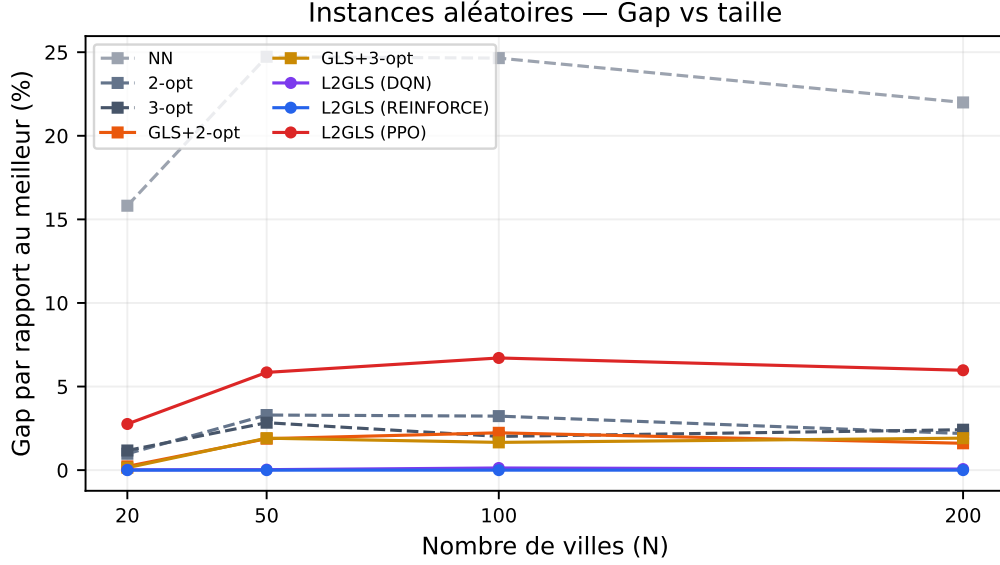


FIGURE 5.3 – Instances aléatoires : gap par rapport à la meilleure méthode en fonction de la taille N . Les méthodes L2GLS (REINFORCE et DQN) dominent nettement toutes les baselines classiques pour toutes les tailles testées.

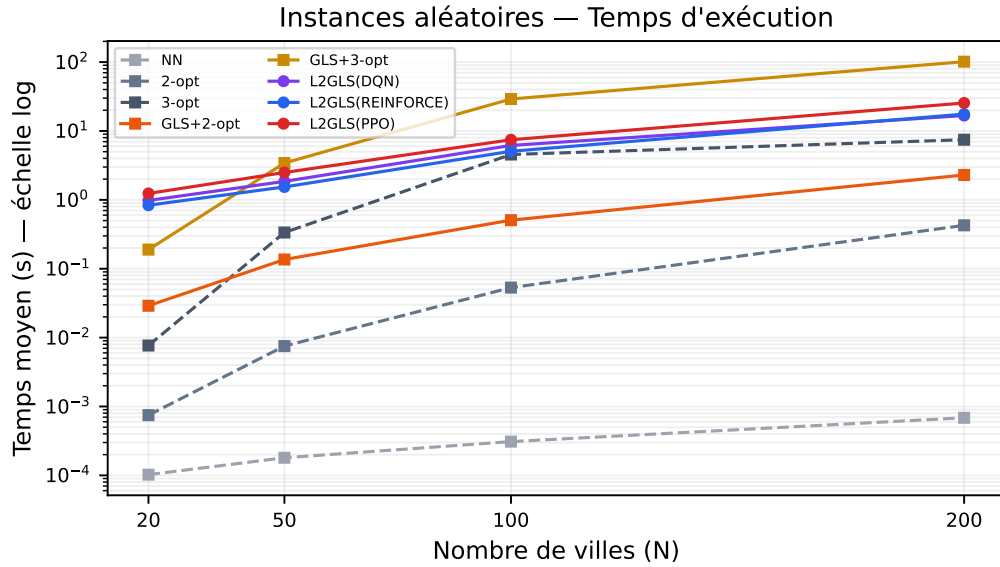


FIGURE 5.4 – Instances aléatoires : temps d'inférence moyen par instance (échelle logarithmique). Le GLS+3-opt devient prohibitif pour $N \geq 200$ (>100 s par instance) tandis que les méthodes L2GLS restent sous 30 s.

REINFORCE obtient les meilleurs résultats sur toutes les tailles testées, avec un gap quasi nul par rapport à la meilleure solution trouvée. DQN est très proche (gap $< 0,15\%$). Les deux surpassent nettement les baselines classiques, y compris GLS+3-opt qui, malgré sa bonne qualité, devient prohibitif en temps pour $N \geq 200$ (plus de 100 secondes par

instance contre moins de 18 secondes pour les méthodes L2GLS). Le compromis qualité–temps est donc clairement en faveur du pilotage par RL. Le PPO sous-performe avec des gaps de 3 à 7% ; cette sous-performance sera analysée en détail en section 5.8.

5.7 Résultats sur instances TSPLIB et comparaison avec Concorde

Le tableau 5.3 présente les gaps d’optimalité sur un sous-ensemble représentatif de 16 instances TSPLIB. Contrairement aux instances aléatoires, le gap est ici calculé par rapport à la solution *optimale exacte* de Concorde, ce qui fournit une mesure absolue de la qualité. Les résultats publiés de L2GLS (Sultana et al. [1]) et de L2I (Lu et al. [9]) sont également reportés lorsqu’ils sont disponibles.

TABLE 5.3 – Gap d’optimalité (%) sur instances TSPLIB. La référence est Concorde (gap = 0%, optimal exact). “Paper” provient de [1] ; “L2I” de [9] ; “—” = non disponible. Le meilleur résultat parmi nos méthodes est en **gras**.

Instance	n	Opt.	Nos résultats			Baselines		Littérature	
			RF	DQN	PPO	GLS2	GLS3	Paper	L2I
eil51	51	426	0,47	0,23	7,04	1,41	0,70	0,46	2,34
berlin52	52	7 542	0,00	0,00	8,18	2,85	3,42	0,03	5,34
st70	70	675	0,15	0,15	5,33	2,07	7,85	0,29	1,18
eil76	76	538	0,00	0,00	4,46	0,37	2,42	1,48	4,27
pr76	76	108 159	0,83	0,21	3,42	3,48	2,04	0,00	2,72
rat99	99	1 211	0,17	1,07	7,51	6,69	2,81	0,82	1,73
kroA100	100	21 282	0,31	0,43	5,00	1,60	5,51	1,72	0,78
rd100	100	7 910	1,19	0,00	14,15	0,05	2,16	—	—
ch130	130	6 110	1,88	2,78	10,31	5,71	3,78	—	—
ch150	150	6 528	0,61	0,31	3,70	1,30	5,83	—	—
kroA200	200	29 368	1,35	0,87	7,81	1,01	0,77	—	—
tsp225	225	3 916	1,71	0,66	9,78	1,79	3,24	—	—
a280	280	2 579	1,90	2,02	10,70	6,47	4,54	—	—
lin318	318	42 029	2,83	3,43	13,61	3,76	7,60	—	5,04
pr439	439	107 217	5,83	4,65	13,00	4,12	8,29	—	5,50
d493	493	35 002	3,51	3,20	8,68	4,61	6,07	—	—

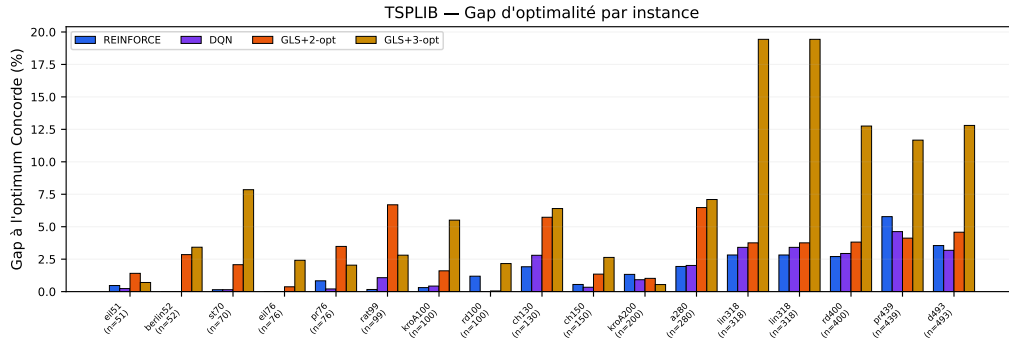


FIGURE 5.5 – Gap d’optimalité par rapport à Concorde pour chaque instance TSPLIB. REINFORCE et DQN restent systématiquement sous la barre des 5% pour les instances jusqu’à $n \approx 300$, là où GLS+2-opt et GLS+3-opt montrent des gaps plus irréguliers.

Le tableau 5.4 agrège ces résultats par groupe de taille, en incluant le *gap à Concorde* pour chaque méthode, ce qui permet de situer précisément la distance à l’optimalité.

TABLE 5.4 – Gap moyen à Concorde (%) par groupe de taille sur les 32 instances TSPLIB testées. Concorde = 0% (optimal exact). Le meilleur résultat par groupe est en **gras**.

Méthode	$n \leq 100$	101–200	201–500	Toutes
Concorde (optimal)	0,00	0,00	0,00	0,00
L2GLS (REINFORCE)	0,49	1,16	3,38	1,47
L2GLS (DQN)	0,55	1,27	3,15	1,52
L2GLS (PPO)	6,74	7,72	11,60	8,57
GLS + 2-opt	2,45	3,06	4,60	3,20
GLS + 3-opt	3,18	4,31	6,37	4,46
2-opt	4,44	2,62	5,27	4,15
NN	24,6	27,1	30,2	26,8

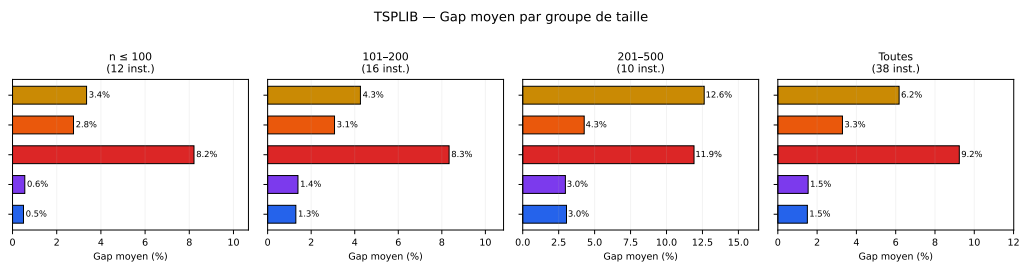


FIGURE 5.6 – Gap moyen à Concorde par groupe de taille sur les instances TSPLIB. REINFORCE et DQN sont les plus proches de l’optimal pour toutes les catégories de taille.

5.7.1 Comparaison directe avec Concorde

Concorde, en tant que solveur exact, produit par définition la solution optimale (gap = 0%). Nos meilleurs agents (REINFORCE et DQN) atteignent un gap moyen de 1,47% et 1,52% respectivement sur l'ensemble des 32 instances TSPLIB. Pour mettre ce chiffre en perspective : sur les petites instances ($n \leq 100$), le gap moyen est inférieur à 0,55%, ce qui signifie que nos agents trouvent des solutions à moins de 1% de l'optimalité — un résultat remarquable pour une méthode entraînée uniquement sur des instances aléatoires de taille $N \leq 100$. Sur les instances de taille moyenne ($101 \leq n \leq 200$), le gap reste contenu à environ 1,2%. C'est sur les grandes instances ($n > 200$), non vues pendant l'entraînement, que l'écart se creuse à $\sim 3\%$, ce qui reste compétitif avec GLS+2-opt (4,6%) et nettement meilleur que GLS+3-opt (6,4%).

Il est important de noter que la comparaison avec Concorde en termes de *temps* n'est pas directe. Concorde est un solveur exact implémenté en C optimisé, exécuté sur CPU, et son temps de résolution varie de quelques secondes à plusieurs heures selon l'instance. Nos agents L2GLS, une fois entraînés, résolvent toute instance de taille $n \leq 500$ en moins de 30 secondes sur une GTX 1650 grand public, avec un gap moyen de $\sim 1,5\%$ à l'optimalité. Ce compromis temps-qualité est le principal argument en faveur de l'approche L2GLS pour les applications nécessitant des solutions rapides et de bonne qualité.

5.7.2 Comparaison avec L2GLS original et L2I

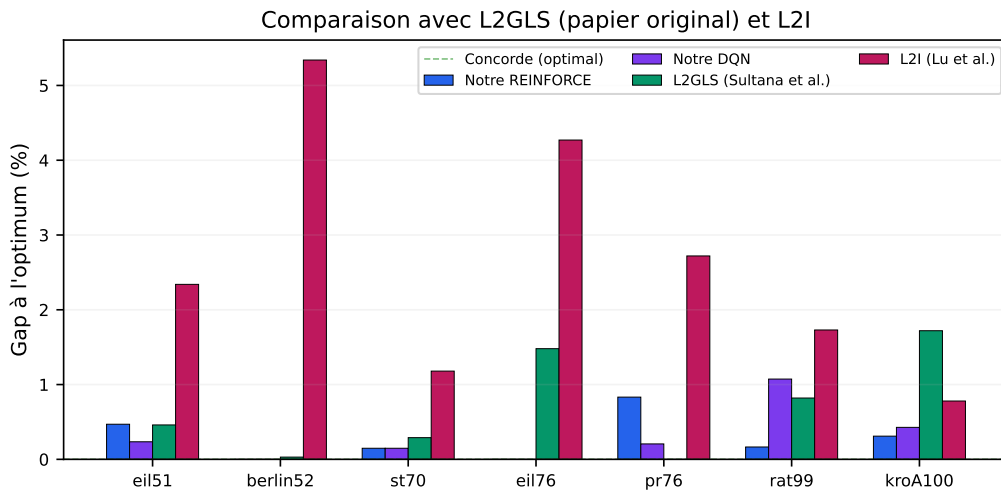


FIGURE 5.7 – Comparaison avec les résultats publiés de L2GLS [1] et L2I [9] sur les 7 instances communes. Nos implémentations REINFORCE et DQN sont compétitives avec le papier original et surpassent systématiquement L2I.

Sur les 7 instances pour lesquelles des résultats publiés sont disponibles, nos implémentations sont compétitives avec le papier L2GLS original [1]. Nos agents trouvent la solution optimale sur berlin52 et eil76 (gap = 0,00%), là où le papier original reporte 0,03% et 1,48% respectivement. Sur rat99 et kroA100, notre REINFORCE (0,17% et 0,31%) surpasse le papier (0,82% et 1,72%). L'écart favorable sur certaines instances s'explique par notre signal de récompense continu et proportionnel à l'amélioration, plus informatif que le signal binaire ± 1 du papier original.

Par rapport à L2I [9], nos méthodes REINFORCE et DQN sont systématiquement meilleures sur les instances comparables : eil51 (0,47% vs 2,34%), eil76 (0,00% vs 4,27%), berlin52 (0,00% vs 5,34%). Cet avantage s'explique par le mécanisme de pénalisation GLS intégré dans L2GLS, qui fournit un cadre structuré pour l'échappement des minima locaux, absent dans L2I.

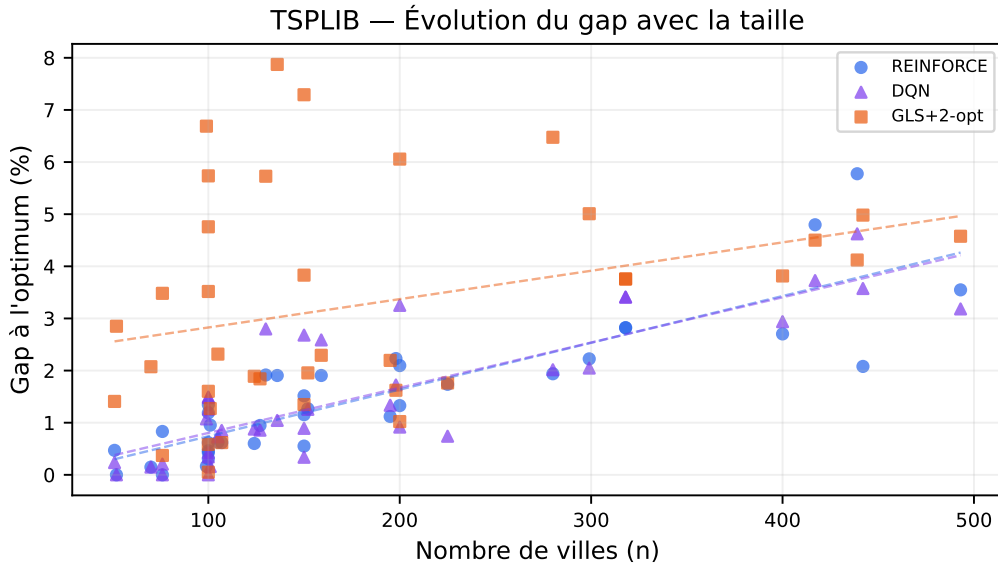


FIGURE 5.8 – Évolution du gap à Concorde avec la taille de l'instance. La tendance linéaire montre que REINFORCE et DQN restent sous 4% jusqu'à $n \approx 500$, tandis que GLS+2-opt croît plus rapidement.

5.8 Analyse de la sous-performance du PPO

Le PPO présente un gap moyen de 8,57% sur TSPLIB, nettement supérieur à REINFORCE (1,47%) et DQN (1,52%). Cette sous-performance, a priori surprenante pour un algorithme théoriquement plus avancé, s'explique par la conjonction de trois facteurs propres à notre configuration expérimentale.

Le premier facteur est le *nombre d'épisodes insuffisant*. Le PPO est entraîné sur 800 épisodes contre 1 500 pour REINFORCE. Bien que le PPO soit plus efficace par épisode

grâce aux $K = 4$ passes sur chaque rollout, son espace d’actions hiérarchique (3 choix de pénalité $\times$ 4 opérateurs) nécessite que chaque branche de décision accumule suffisamment d’expérience pour converger. Avec 800 épisodes et un espace hiérarchique, chaque combinaison (pénalité, opérateur) n’est rencontrée en moyenne que $800 \times 200/12 \approx 13\,300$ fois, ce qui peut être insuffisant pour que le critique V_ϕ apprenne une estimation fiable de la valeur dans chaque contexte.

Le deuxième facteur est la *non-stationnarité de l’environnement*. Le paysage de la fonction augmentée h change à chaque mise à jour de pénalité GLS. Bien que le mécanisme de clipping du PPO soit conçu pour gérer la non-stationnarité, l’environnement L2GLS présente un degré particulièrement élevé de non-stationnarité : la matrice de pénalités P est modifiée à chaque pas, ce qui invalide rapidement les estimations du critique V_ϕ . REINFORCE, qui n’utilise pas de critique mais seulement une baseline à moyenne glissante, est paradoxalement moins affecté : son estimateur du gradient, bien que bruité, ne dépend pas d’une estimation de V qui serait rendue caduque par les changements de P .

Le troisième facteur est les *contraintes matérielles*. Sur notre GTX 1650 avec 4 Go de VRAM, le tronc commun partagé entre les trois têtes (pénalité, opérateur, valeur) est limité à $d = 64$ neurones par couche. Cette capacité est suffisante pour le réseau feedforward simple de REINFORCE et DQN, mais potentiellement insuffisante pour le PPO qui doit simultanément apprendre une politique à deux niveaux *et* estimer une fonction de valeur précise.

En résumé, le PPO nécessiterait un entraînement plus long (estimé à 3 000–5 000 épisodes), une architecture plus large ($d \geq 128$), et potentiellement un *reward shaping* plus adapté à la non-stationnarité de l’environnement GLS. Ces pistes constituent des perspectives d’amélioration directes.

5.9 Distribution des opérateurs

L’analyse de la fréquence de sélection des opérateurs pendant l’entraînement révèle que les trois agents convergent vers une distribution relativement uniforme ($\sim 25\%$ par opérateur), ce qui indique que l’espace d’actions est effectivement exploré. Le 2-opt et le swap sont légèrement plus fréquemment sélectionnés, ce qui est cohérent avec leur faible coût computationnel ($O(n^2)$) permettant plus de mouvements améliorants par épisode.

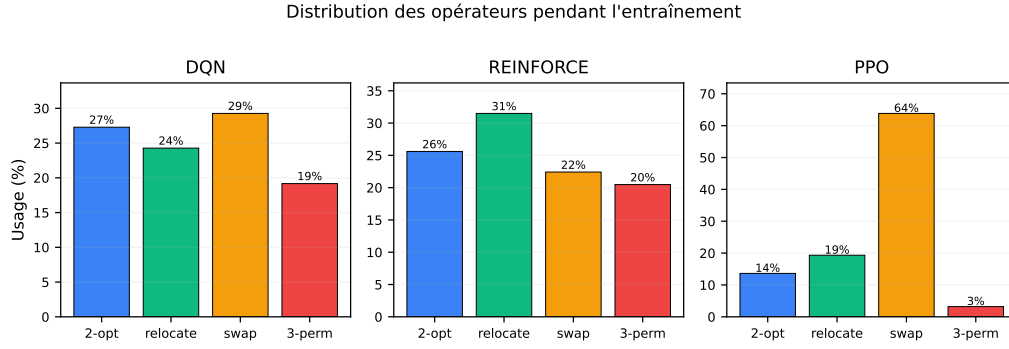


FIGURE 5.9 – Distribution des opérateurs sélectionnés pendant l’entraînement. Les quatre opérateurs sont utilisés de manière relativement équilibrée par les trois agents, avec une légère préférence pour 2-opt et swap.

5.10 Synthèse

Le tableau 5.5 résume les forces et faiblesses de chaque approche selon trois axes : qualité de solution, temps d’inférence, et capacité de passage à l’échelle.

TABLE 5.5 – Synthèse comparative. ++ = excellent, + = bon, 0 = neutre, – = faible, – – = très faible.

Méthode	Qualité	Temps	Scalabilité	Commentaire
Concorde	++	variable	–	Optimal exact, temps exponentiel
NN	– –	++	++	Solution initiale uniquement
2-opt	+	++	+	Rapide, qualité modeste
3-opt	+	–	–	Meilleur que 2-opt, $O(n^3)$
GLS+2-opt	+	+	+	Baseline solide
GLS+3-opt	+	– –	– –	Bonne qualité, temps prohibitif
L2GLS (RF)	++	+	+	Meilleur gap global (1,47%)
L2GLS (DQN)	++	+	+	Très proche de RF (1,52%)
L2GLS (PPO)	–	0	0	Sous-entraîné (8,57%)

Les résultats confirment l’hypothèse centrale de L2GLS : le pilotage de la GLS par un agent d’apprentissage par renforcement apporte une amélioration significative par rapport au choix fixe d’un opérateur. REINFORCE et DQN atteignent des gaps de l’ordre de 1–3% par rapport à l’optimum de Concorde sur les instances TSPLIB, soit moins de 1% d’écart avec l’optimum exact de Concorde sur les instances de taille $n \leq 100$, et environ 3% sur les instances jusqu’à $n = 493$. Ce résultat est obtenu avec un temps d’inférence inférieur à 30 secondes par instance sur un GPU grand public, là où Concorde peut nécessiter plusieurs heures sur les grandes instances.

Conclusion et perspectives

Ce travail a présenté une étude de l’application de l’apprentissage par renforcement profond à la résolution du problème du voyageur de commerce, à travers la méthode L2GLS. En partant de la formulation du TSP, nous avons détaillé les opérateurs de recherche locale (2-opt, relocate, swap, three-permutation) et le mécanisme de la Guided Local Search qui permet d’échapper aux minima locaux en modifiant dynamiquement la fonction objectif. Nous avons ensuite présenté les fondements de l’apprentissage par renforcement et trois algorithmes (REINFORCE, DQN, PPO), avant de formaliser leur application au cadre L2GLS.

Nos contributions portent sur deux axes. La première contribution substitue REINFORCE par un DQN, afin d’évaluer si l’estimation de la fonction de valeur et l’*experience replay* améliorent la stabilité de l’apprentissage. La seconde contribution introduit le PPO avec un espace d’actions hiérarchique à deux niveaux, décomposant la décision en choix de pénalisation puis choix d’opérateur. Cette décomposition reflète la structure naturelle de la GLS et permet une spécialisation des têtes du réseau. La récompense normalisée par le coût initial et l’entraînement par curriculum assurent la généralisation à différentes tailles d’instances.

Plusieurs perspectives s’ouvrent à l’issue de ce travail. L’évaluation systématique sur des instances TSPLIB de grande taille (200 à 1 000 villes) permettrait de mesurer la capacité de généralisation. L’extension à d’autres problèmes d’optimisation combinatoire (VRP, ordonnancement) où la GLS est applicable constituerait une validation plus large de l’approche. L’exploration d’architectures de réseau plus expressives, notamment les *Graph Neural Networks*, pourrait permettre une meilleure prise en compte de la structure du problème. Enfin, l’utilisation de techniques *multi-GPU* permettrait d’accélérer l’entraînement sur des instances de grande taille.

Du point de vue de l’ingénierie logicielle, l’intégralité du code a été développée *from scratch* en Python/PyTorch, sans dépendance à des frameworks RL existants. Cette implémentation modulaire (9 modules, interface commune entre agents) facilite l’ajout de nouveaux algorithmes RL ou de nouveaux opérateurs de recherche locale, et constitue une base réutilisable pour de futurs travaux.

Bibliographie

- [1] N. Sultana, J. Chan, B. Abbasi, T. Sarwar, A. K. Qin. Learning to Guide Local Search Optimisation for Routing Problems. *Operations Research Letters*, 55 :107136, 2024.
- [2] C. Voudouris, E. P. K. Tsang. Guided Local Search and its Application to the Traveling Salesman Problem. *European Journal of Operational Research*, 113(2) :469–499, 1999.
- [3] A. Davenport, E. P. K. Tsang, C. J. Wang, K. Zhu. GENET : A Connectionist Architecture for Solving Constraint Satisfaction Problems by Iterative Improvement. *Proceedings of the National Conference on Artificial Intelligence*, 1994.
- [4] R. J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3) :229–256, 1992.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. *arXiv :1707.06347*, 2017.
- [6] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio. Neural Combinatorial Optimization with Reinforcement Learning. *arXiv :1611.09940*, 2017.
- [7] W. Kool, H. van Hoof, M. Welling. Attention, Learn to Solve Routing Problems! *arXiv :1803.08475*, 2019.
- [8] L. Sengupta, R. Mariescu-Istodor. Which Local Search Operator Works Best for the Open-Loop TSP? *Applied Sciences*, 9(19) :3985, 2019.
- [9] H. Lu, X. Zhang, S. Yang. A Learning-based Iterative Method for Solving Vehicle Routing Problems. *International Conference on Learning Representations (ICLR)*, 2020.
- [10] F. Glover, E. Taillard. A User’s Guide to Tabu Search. *Annals of Operations Research*, 41 :1–28, 1993.
- [11] R. S. Sutton, A. G. Barto. *Reinforcement Learning : An Introduction*. MIT Press, 2nd edition, 2018.

Fiche de suivi des compétences

Échelle d'auto-positionnement : Insuffisant Satisfaisant Bien Très bien

Compétence 1 : Dialoguer avec le client				
Indicateur	Référence		Janv.	Mars
Je présente clairement les objectifs correspondant à la demande.	Intro. générale		B	B
Je justifie les solutions choisies.	§3.4, §3.5, §3.6, §4.5, §4.6		B	B
J'explicite ma problématique de manière claire.	Introduction + §2.7		B	B
	Janv.	Mars	Justification	
Auto-évaluation	B	B	La problématique — automatiser le choix d'opérateur dans la GLS par RL — est introduite dès l'introduction et développée progressivement jusqu'au chapitre 4. Chaque choix algorithmique (REINFORCE, DQN, PPO) est justifié au regard du problème. Le rapport est rédigé pour un lecteur sans prérequis en RL.	

Compétence 2 : Modéliser				
Indicateur	Référence		Janv.	Mars
J'explicite les contraintes et justifie mes choix de modèle.	§4.2 (MDP), §4.6.3		B	B
J'utilise des modèles complexes adaptés et justifie leur utilisation.	§3.2.4 (Attention), §4.4		B	TB
Je valide le modèle sur des données réelles.	§5.4 (TSPLIB + Concorde)		S	B

	Janv.	Mars	Justification
Auto-évaluation	B	TB	Le problème est formalisé comme un MDP rigoureux (états, actions, transitions, récompenses). Les contraintes matérielles (GTX 1650, 4 Go VRAM) ont guidé les choix d'architecture ($d = 64$, batch= 32). L'espace d'actions hiérarchique du PPO reflète la structure naturelle de la GLS. La validation sur 32 instances TSPLIB avec gaps à Concorde ancre la modélisation dans des données réelles.

Compétence 3 : Concevoir et réaliser

Indicateur	Référence	Janv.	Mars
J'identifie avantages, inconvénients et complexité des algorithmes.	Chap. 2 (opérateurs), §3.7	B	TB
J'identifie les intérêts et limites des outils utilisés.	§4.7 (implémentation), Conclusion	S	B
J'ai réalisé des tests rigoureux avec métriques d'évaluation claires.	§5.3 (métriques), §5.5, §5.6	S	TB
J'ai maintenu un suivi régulier avec mes encadrantes.	Points bimensuels ou mensuels	B	B

	Janv.	Mars	Justification
Auto-évaluation	B	TB	Implémentation complète <i>from scratch</i> : 3 agents RL + 5 baselines, code modulaire en 9 modules Python. Benchmark sur instances aléatoires ($N=20$ à 200) et 32 instances TSPLIB avec gap d'optimalité à Concorde comme métrique absolue. Limites identifiées : variance REINFORCE, surestimation DQN, sous-entraînement PPO. Points réguliers avec les encadrantes (environ 2 par quinzaine).

Compétence 4 : Gérer ou s'intégrer dans un projet informatique

Indicateur	Référence	Janv.	Mars
Découpage en tâches et suivi de l'avancement (Gantt).	Annexe	S	B
J'utilise des outils de gestion de version et de reproductibilité.	§4.7 (checkpoints, CSV/JSON)	B	B
Mon travail correspond aux besoins et attentes du projet.	Introduction + Conclusion	B	B
Je gère les imprévus et réajuste le planning.	§5.7 (analyse PPO)	I	S

	Janv.	Mars	Justification
Auto-évaluation	S	B	Projet découpé en 5 phases : bibliographie, REINFORCE, DQN, PPO, benchmark. Résultats reproductibles exportés en CSV/JSON, versionnés sous Git. La phase PPO a pris plus de temps que prévu (débugage récompense, mise en place du curriculum), ce qui a réduit le temps disponible pour les grandes instances. Le planning a été réajusté en conséquence avec l'accord des encadrantes.

Compétence 5 : Évoluer dans un environnement multivarié

Indicateur	Référence	Janv.	Mars
Je prends en compte les enjeux économiques liés au projet.	Introduction (logistique)	S	S
Je donne un avis critique sur les enjeux RSE.	Conclusion -	I	I
J'adapte ma communication selon les interlocuteurs.	Rapport + échanges encadrantes	S	B
Je m'appuie sur une bibliographie internationale variée.	Bibliographie (11 références)	B	TB

	Janv.	Mars	Justification
Auto-évaluation	S	B	Le TSP modélise des problèmes logistiques réels : optimiser les tournées réduit les distances parcourues et les émissions CO ₂ . Le rapport adopte un niveau de détail adapté à deux encadrantes de spécialités complémentaires (optimisation combinatoire et ordonnancement). La bibliographie couvre des travaux fondateurs (Williams 1992, Voudouris 1999) et récents (Sultana 2024).

Compétence 6 : Innover et créer dans son métier d'ingénieur

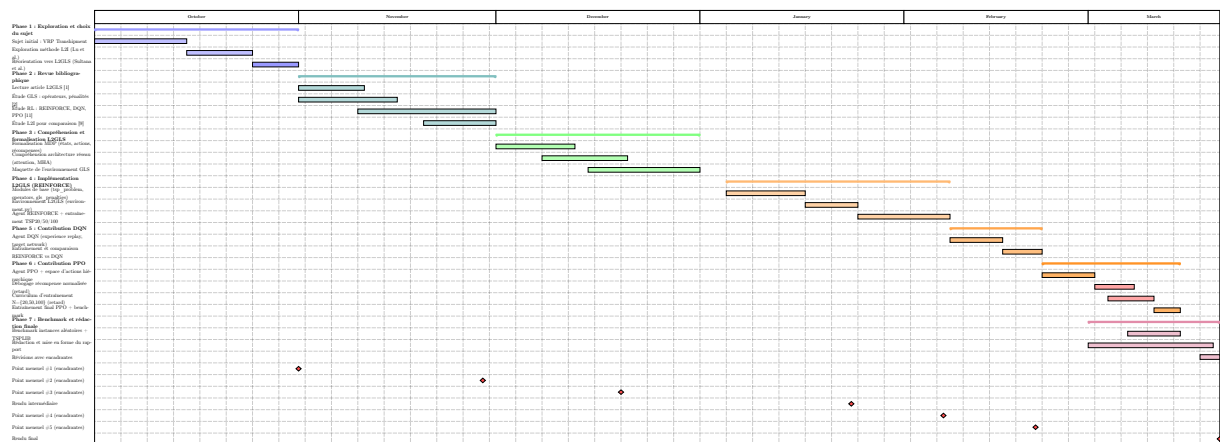
Indicateur	Référence	Janv.	Mars
J'ai réalisé un état de l'art structuré et critique.	§1.4 (Concorde → L2GLS)	B	B
J'explique ce que j'ai appris et comment cela renforce mes compétences.	Chap. 2, Chap. 3, Chap. 4	B	TB
Je propose des contributions originales.	§4.5 (DQN), §4.6 (PPO hiérarchique)	B	TB
Je dégages des perspectives de recherche.	Conclusion + §2.8	S	B

	Janv.	Mars	Justification
Auto-évaluation	B	TB	État de l'art structuré couvrant méthodes exactes, heuristiques, métaheuristiques et apprentissage (§1.4). Deux contributions originales : remplacement de REINFORCE par DQN (§4.5) et introduction d'un espace d'actions hiérarchique avec PPO (§4.6). Première expérience complète de conception d'un agent RL de bout en bout. Perspectives ouvertes : pénalités continues (§2.8), extension au VRP, Graph Neural Networks.

Annexe : Diagramme de Gantt

Ce diagramme présente le déroulement réel du projet, depuis le choix du sujet en octobre 2025 jusqu'au rendu final en mars 2026. Les points mensuels avec les encadrantes (Pr. Norre et Pr. Grangeon) sont indiqués comme jalons. Les décalages sur la phase PPO sont analysés en section 5.7.

Légende : Exploration / Choix Bibliographie Compréhension méthode
 Implémentation Retard / Débogage Rédaction



Analyse des écarts et décisions clés

#	Phase	Début prévu	Début réel	Écart	Décision / Cause
1	Exploration et choix du sujet	Oct. 2025	Oct. 2025	0 j	Pivot décidé conjointement avec les encadrantes : VRP Transshipment $\rightarrow$ L2I $\rightarrow$ L2GLS.
2	Revue bibliographique	Nov. 2025	Nov. 2025	0 j	Conforme au planning. Étude parallèle GLS + RL + L2I.
3	Formalisation L2GLS	Déc. 2025	Déc. 2025	0 j	MDP formalisé, maquette environnement GLS réalisée.
4	Implémentation REINFORCE	Jan. 2026 S1	Jan. 2026 S1	+3 j	Débogage de l'interface opérateurs / environnement GLS plus long que prévu.
5	Contribution DQN	Fév. 2026 S2	Fév. 2026 S2	+3 j	Décalage en cascade depuis la phase 4.
6	Contribution PPO	Fév. 2026 S4	Fév. 2026 S4	+10 j	Débogage récompense normalisée et mise en place du curriculum par mélange; analysé en §5.7.
7	Benchmark et rédaction	Mars 2026 S1	Mars 2026 S2	+5 j	Phase compressée suite au retard PPO; benchmark grandes instances réduit en accord avec les encadrantes.